이 파트에서는 **딥러닝(Deep Learning)**의 핵심 개념을 학습합니다.

인공 신경망의 기본 원리부터 시작하여, 이미지 처리에 강한 **CNN(합성곱 신경망)**, 시퀀스 데이터를 다루는 **RNN(순환 신경망)**, 그리고 생성형 AI의 기반이 되는 **자연어 처리(NLP)** 기초까지 다룹니다.

학습 목표

- 퍼셉트론, 활성화 함수, 역전파 등 신경망의 핵심 원리를 이해합니다.
- CNN의 합성곱 연산과 이미지 분류 원리를 설명할 수 있습니다.
- RNN/LSTM의 시퀀스 처리 방식과 한계를 이해합니다.
- 토큰화, 워드 임베딩 등 NLP 기초 개념을 익힙니다.

이 파트의 구성

챕터	제목	핵심 내용
Chapter 4	신경망의 기초	퍼셉트론, 순전파, 역전파, 경사하강법
Chapter 5	CNN (합성곱 신경망)	합성곱, 풀링, 대표 모델(LeNet~ResNet)
Chapter 6	RNN과 자연어 처리 기초	LSTM, GRU, 토큰화, Word2Vec

지금까지 머신러닝의 기본 개념과 다양한 알고리즘을 살펴보았습니다. 이번 챕터에서는 현대 인공지능의 핵심인 **신경망(Neural Network)**에 대해 깊이 있게 다룹니다. 신경망은 인간의 뇌 구조에서 영감을 받아 설계된 모델로, 이미지 인식, 자연어 처리, 게임 플레이 등 수많은 분야에서 놀라운 성과를 거두고 있습니다.

이 챕터를 마치면 신경망이 어떻게 학습하는지 그 전체 흐름을 이해하고, 직접 간단한 신경망을 구현해 손글씨 숫자를 분류할 수 있게 됩니다.

> 💡 ***학습 목표*** *- 생물학적 뉴런에서 인공 뉴런의 개념이 어떻게 유래했는지 설명할 수 있습니다. - 퍼셉트론과 다층 퍼셉트론(MLP)의 구조를 이해합니다. - 주요 활성화 함수의 특징과 용도를 구분합니다. - 순전파, 손실함수, 역전파, 경사하강법으로 이어지는 학습 파이프라인을 이해합니다. - 최적화 기법과 과적합 방지 기법을 적용할 수 있습니다. - PyTorch로 MNIST 손글씨 분류 신경망을 구현합니다.*

4.1 생물학적 뉴런에서 인공 뉴런으로

생물학적 뉴런의 구조

우리 뇌에는 약 860억 개의 **뉴런(Neuron)**이 있습니다. 각 뉴런은 다른 뉴런으로부터 신호를 받아 처리하고, 그 결과를 다음 뉴런으로 전달하는 역할을 합니다.

생물학적 뉴런의 주요 구성 요소는 다음과 같습니다.

- **수상돌기(Dendrite)**: 다른 뉴런으로부터 신호를 수신합니다.
- **세포체(Cell Body)**: 수신한 신호를 합산하고 처리합니다.
- **축삭돌기(Axon)**: 처리된 신호를 다음 뉴런으로 전달합니다.
- **시냅스(Synapse)**: 뉴런 간의 연결 부위로, 신호의 강도를 조절합니다.

뉴런은 입력 신호의 합계가 일정 **임계값(Threshold)**을 넘을 때만 활성화(발화)하여 다음 뉴런으로 신호를 전달합니다.

인공 뉴런의 탄생

1943년 워런 맥컬럭(Warren McCulloch)과 월터 피츠(Walter Pitts)는 생물학적 뉴런을 수학적으로 모델링한 **인공 뉴런(Artificial Neuron)**을 제안했습니다.

인공 뉴런의 동작 원리는 다음과 같습니다.

1. 여러 입력값 $x_1, x_2, \ldots, x_n$을 받습니다.

2. 각 입력에 **가중치(Weight)** $w_1, w_2, \ldots, w_n$를 곱합니다.
3. 가중합(Weighted Sum)에 **편향(Bias)** b를 더합니다.
4. **활성화 함수(Activation Function)**를 통과시켜 최종 출력을 냅니다.

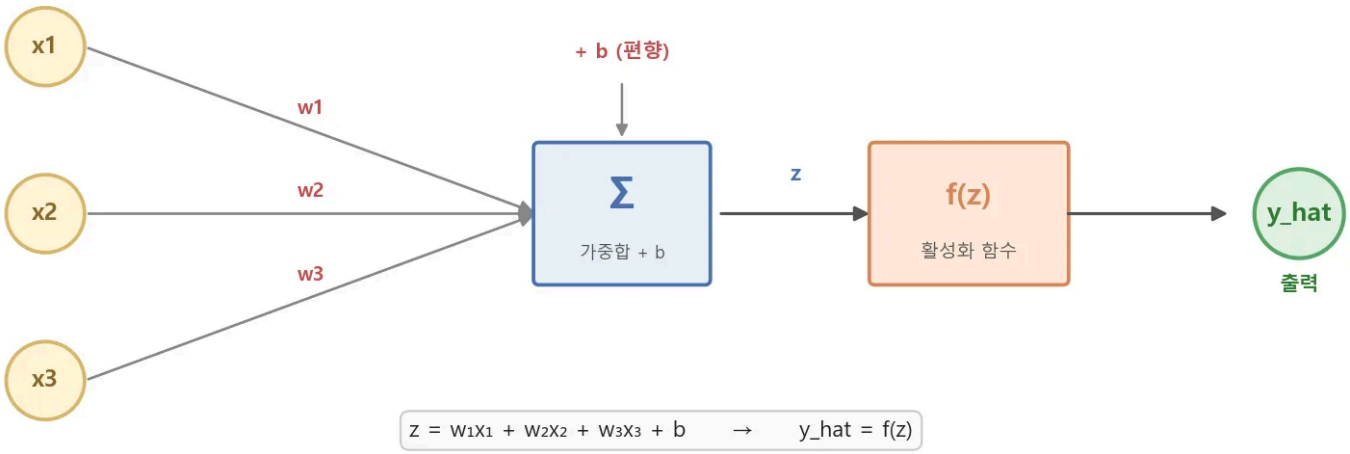
수식으로 표현하면 다음과 같습니다.

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \mathbf{w}^T \mathbf{x} + b$$

$$\text{output} = f(z)$$

여기서 f는 활성화 함수입니다.

인공 뉴런(Artificial Neuron)의 동작 원리



> 📌 ***생물학적 뉴런 vs 인공 뉴런*** *| 생물학적 뉴런 | 인공 뉴런 | |---|---| | 수상돌기 | 입력값 x_i | | 시냅스 연결 강도 | 가중치 w_i | | 세포체 신호 합산 | 가중합 계산 | | 임계값 초과 시 발화 | 활성화 함수 | | 축삭돌기 신호 전달 | 출력값 |*

4.2 퍼셉트론과 다층 퍼셉트론(MLP)

퍼셉트론 (Perceptron)

1957년 프랑크 로젠블라트(Frank Rosenblatt)가 제안한 **퍼셉트론(Perceptron)**은 최초의 학습 가능한 인공 신경망입니다. 퍼셉트론은 입력층과 출력층 단 두 개의 층으로 구성됩니다.

퍼셉트론의 출력은 다음과 같습니다.

$$\hat{y} = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

퍼셉트론은 AND, OR 같은 선형 분리 가능한 문제는 풀 수 있지만, XOR 문제처럼 선형으로 분리되지 않는 문제는 풀 수 없다는 한계가 있습니다.

> ⚠️ ***퍼셉트론의 한계*** *1969년 마빈 민스키(Marvin Minsky)와 시모어 페이퍼트(Seymour Papert)는 단층 퍼셉트론이 XOR 문제를 해결할 수 없음을 수학적으로 증명했습니다. 이 발견은 한동안 인공지능 연구를 침체시켰지만, 다층 구조를 통해 극복할 수 있었습니다.*

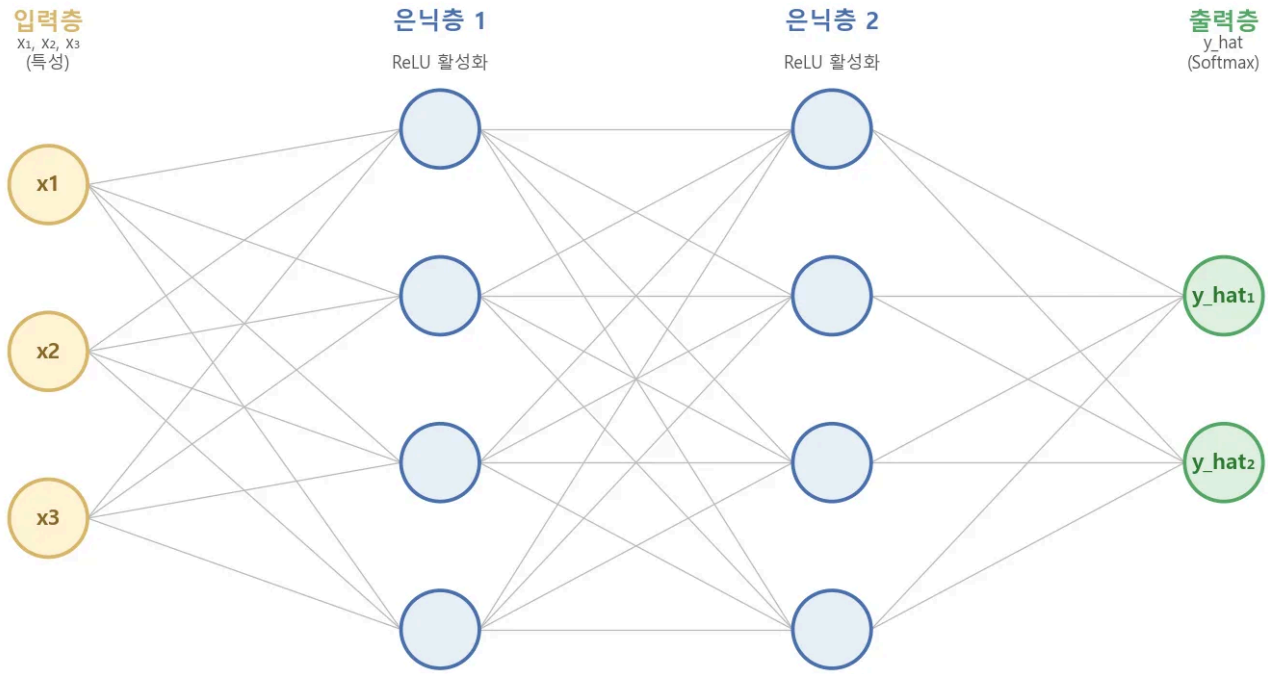
다층 퍼셉트론 (MLP, Multi-Layer Perceptron)

퍼셉트론의 한계를 극복하기 위해 **은닉층(Hidden Layer)**을 추가한 **다층 퍼셉트론(MLP)**이 등장했습니다.

MLP는 다음 세 가지 층으로 구성됩니다.

- **입력층(Input Layer)**: 데이터의 특성(Feature)을 받아들이는 층입니다.
- **은닉층(Hidden Layer)**: 입력층과 출력층 사이에서 데이터를 변환합니다. 여러 개를 쌓을 수 있습니다.
- **출력층(Output Layer)**: 최종 예측값을 출력하는 층입니다.

다층 퍼셉트론(MLP) 구조 — 입력층·은닉층·출력층



각 층의 모든 뉴런이 다음 층의 모든 뉴런과 연결되는 완전 연결층(Fully Connected) — Dense 층이라고도 부릅니다.

각 층의 뉴런은 이전 층의 모든 뉴런과 연결되어 있으며, 이를 **완전 연결층(Fully Connected Layer)** 또는 **Dense 층**이라고 합니다.

은닉층을 여러 겹 쌓은 신경망을 **심층 신경망(Deep Neural Network, DNN)**이라고 하며, 이것이 **딥러닝(Deep Learning)**의 어원입니다.

> 💡 ***왜 은닉층이 필요한가?*** *은닉층은 데이터에서 점점 더 추상적인 특징을 학습합니다. 예를 들어 이미지 인식에서 첫 번째 은닉층은 선과 모서리를, 두 번째는 도형을, 세 번째는 얼굴의 부위를 학습하는 식으로 계층적 특징 추출이 이루어집니다.*

4.3 활성화 함수 (Sigmoid, ReLU, Softmax)

활성화 함수는 신경망에 **비선형성(Non-linearity)**을 부여합니다. 활성화 함수 없이 가중합만 계산하면, 아무리 층을 깊게 쌓아도 결국 하나의 선형 변환과 동일해집니다.

Sigmoid 함수

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid 함수는 출력값을 0과 1 사이로 압축합니다. 이진 분류의 출력층에서 주로 사용됩니다.

특징	내용
출력 범위	(0, 1)
주요 용도	이진 분류 출력층
장점	확률 해석 가능
단점	기울기 소실(Vanishing Gradient) 문제 발생

> ⚠️ ***기울기 소실 문제*** *Sigmoid는 입력값이 매우 크거나 작을 때 기울기가 거의 0이 됩니다. 역전파 시 기울기가 점점 작아져 초기 층의 학습이 매우 느려지는* ***기울기 소실(Vanishing Gradient)*** *문제가 발생합니다.*

ReLU 함수 (Rectified Linear Unit)

$$\text{ReLU}(z) = \max(0, z)$$

ReLU는 현재 가장 널리 사용되는 활성화 함수입니다. 입력이 0보다 크면 그대로 출력하고, 0 이하면 0을 출력합니다.

특징	내용
출력 범위	$[0, +\infty)$
주요 용도	은닉층
장점	계산이 빠르고, 기울기 소실 문제 완화
단점	음수 입력에서 뉴런이 죽는 Dying ReLU 문제

ReLU의 변형으로는 음수 영역에 작은 기울기를 주는 **Leaky ReLU**, 부드러운 곡선 형태의 **ELU**, **GELU** 등이 있습니다.

Softmax 함수

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Softmax는 다중 분류 문제의 출력층에서 사용됩니다. 각 클래스의 출력값을 확률로 변환하며, 모든 출력의 합은 1이 됩니다.

예를 들어 숫자 0~9를 분류하는 경우, Softmax는 10개 클래스 각각의 확률을 반환합니다.

> 📌 ***활성화 함수 선택 가이드*** *| 위치 | 권장 함수 | |---|---| | 은닉층 (일반) | ReLU 또는 Leaky ReLU | | 이진 분류 출력층 | Sigmoid | | 다중 분류 출력층 | Softmax | | 회귀 출력층 | 없음 (Linear) |*

활성화 함수 시각화 코드

```
import numpy as np
import matplotlib.pyplot as plt

z = np.linspace(-5, 5, 200)

# Sigmoid
sigmoid = 1 / (1 + np.exp(-z))

# ReLU
relu = np.maximum(0, z)

# Softmax는 벡터 입력이므로 간단한 예시
z_vec = np.array([2.0, 1.0, 0.5])
softmax = np.exp(z_vec) / np.sum(np.exp(z_vec))
print("Softmax 출력:", softmax)  # [0.659, 0.242, 0.099]

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].plot(z, sigmoid, label='Sigmoid', color='blue')
axes[0].set_title('Sigmoid')
axes[0].grid(True)
axes[0].axhline(0, color='k', linewidth=0.5)
axes[0].axvline(0, color='k', linewidth=0.5)

axes[1].plot(z, relu, label='ReLU', color='red')
axes[1].set_title('ReLU')
axes[1].grid(True)
axes[1].axhline(0, color='k', linewidth=0.5)
axes[1].axvline(0, color='k', linewidth=0.5)

plt.tight_layout()
plt.show()
```

위 코드를 실행하면 두 활성화 함수의 형태를 비교할 수 있습니다.

4.4 순전파 (Forward Propagation)

순전파(Forward Propagation)는 입력 데이터가 신경망의 입력층에서 출력층까지 차례로 계산되어 예측값을 만들어내는 과정입니다.

순전파 계산 과정

2개의 은닉층을 가진 신경망을 예로 들어봅니다.

층별 계산:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}$$ $$\mathbf{a}^{(1)} = f(\mathbf{z}^{(1)})$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)} \mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$ $$\mathbf{a}^{(2)} = f(\mathbf{z}^{(2)})$$

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{W}^{(3)} \mathbf{a}^{(2)} + \mathbf{b}^{(3)})$$

여기서 $\mathbf{W}^{(l)}$는 l번째 층의 가중치 행렬, $\mathbf{b}^{(l)}$는 편향 벡터입니다.

NumPy로 구현하는 순전파

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def relu(z):
    return np.maximum(0, z)

def softmax(z):
    exp_z = np.exp(z - np.max(z))  # 수치 안정성을 위해 최댓값을 뺌
    return exp_z / np.sum(exp_z)

# 간단한 신경망 파라미터 초기화 (입력 2, 은닉 3, 출력 2)
np.random.seed(42)
W1 = np.random.randn(3, 2) * 0.1  # (은닉 뉴런 수, 입력 크기)
b1 = np.zeros(3)
W2 = np.random.randn(2, 3) * 0.1  # (출력 크기, 은닉 뉴런 수)
b2 = np.zeros(2)

# 입력 데이터
x = np.array([1.0, 2.0])

# 순전파
z1 = W1 @ x + b1          # 은닉층 가중합
a1 = relu(z1)              # 은닉층 활성화

z2 = W2 @ a1 + b2          # 출력층 가중합
y_hat = softmax(z2)        # 출력층 활성화 (확률)

print("예측 확률:", y_hat)
```

순전파는 계산 방향이 입력 → 출력으로 한 방향이기 때문에 매우 빠릅니다. 학습이 끝난 후 새로운 데이터에 대한 예측도 순전파만으로 수행합니다.

4.5 손실함수 (MSE, Cross-Entropy)

손실함수(Loss Function)는 모델의 예측값과 실제 정답 사이의 차이를 수치로 나타냅니다. 신경망의 목표는 이 손실값을 최소화하는 것입니다.

평균 제곱 오차 (MSE, Mean Squared Error)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

MSE는 주로 **회귀(Regression)** 문제에 사용됩니다. 예측값과 실제값의 차이를 제곱하여 평균을 냅니다. 제곱을 취하기 때문에 큰 오차에 더 큰 패널티를 부여합니다.

```
import numpy as np

def mse_loss(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

y_true = np.array([3.0, 5.0, 2.5])
y_pred = np.array([2.8, 4.5, 2.7])

loss = mse_loss(y_true, y_pred)
print(f"MSE 손실: {loss:.4f}")  # MSE 손실: 0.0933
```

교차 엔트로피 손실 (Cross-Entropy Loss)

$$\text{Cross-Entropy} = -\sum_{c=1}^{K} y_c \log(\hat{y}_c)$$

교차 엔트로피(Cross-Entropy)는 **분류(Classification)** 문제에 사용됩니다. 정답 클래스의 예측 확률이 낮을수록 손실이 크게 증가합니다.

이진 분류의 경우 **이진 교차 엔트로피(Binary Cross-Entropy)**를 사용합니다.

$$\text{BCE} = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

```
import numpy as np

def cross_entropy_loss(y_true, y_pred):
    # 수치 안정성을 위해 클리핑
    y_pred = np.clip(y_pred, 1e-12, 1 - 1e-12)
    return -np.sum(y_true * np.log(y_pred))

# 정답: 클래스 1 (원-핫 인코딩)
y_true = np.array([0, 1, 0])

# 좋은 예측 (정답 클래스 확률이 높음)
y_pred_good = np.array([0.1, 0.8, 0.1])
# 나쁜 예측 (정답 클래스 확률이 낮음)
y_pred_bad  = np.array([0.4, 0.2, 0.4])

print(f"좋은 예측의 손실: {cross_entropy_loss(y_true, y_pred_good):.4f}")  # 0.2231
print(f"나쁜 예측의 손실: {cross_entropy_loss(y_true, y_pred_bad):.4f}")   # 1.6094
```

> 📌 ***손실함수 선택 기준*** *| 문제 유형 | 권장 손실함수 | |---|---| | 회귀 | MSE (Mean Squared Error) | | 이진 분류 | Binary Cross-Entropy | | 다중 클래스 분류 | Categorical Cross-Entropy |*

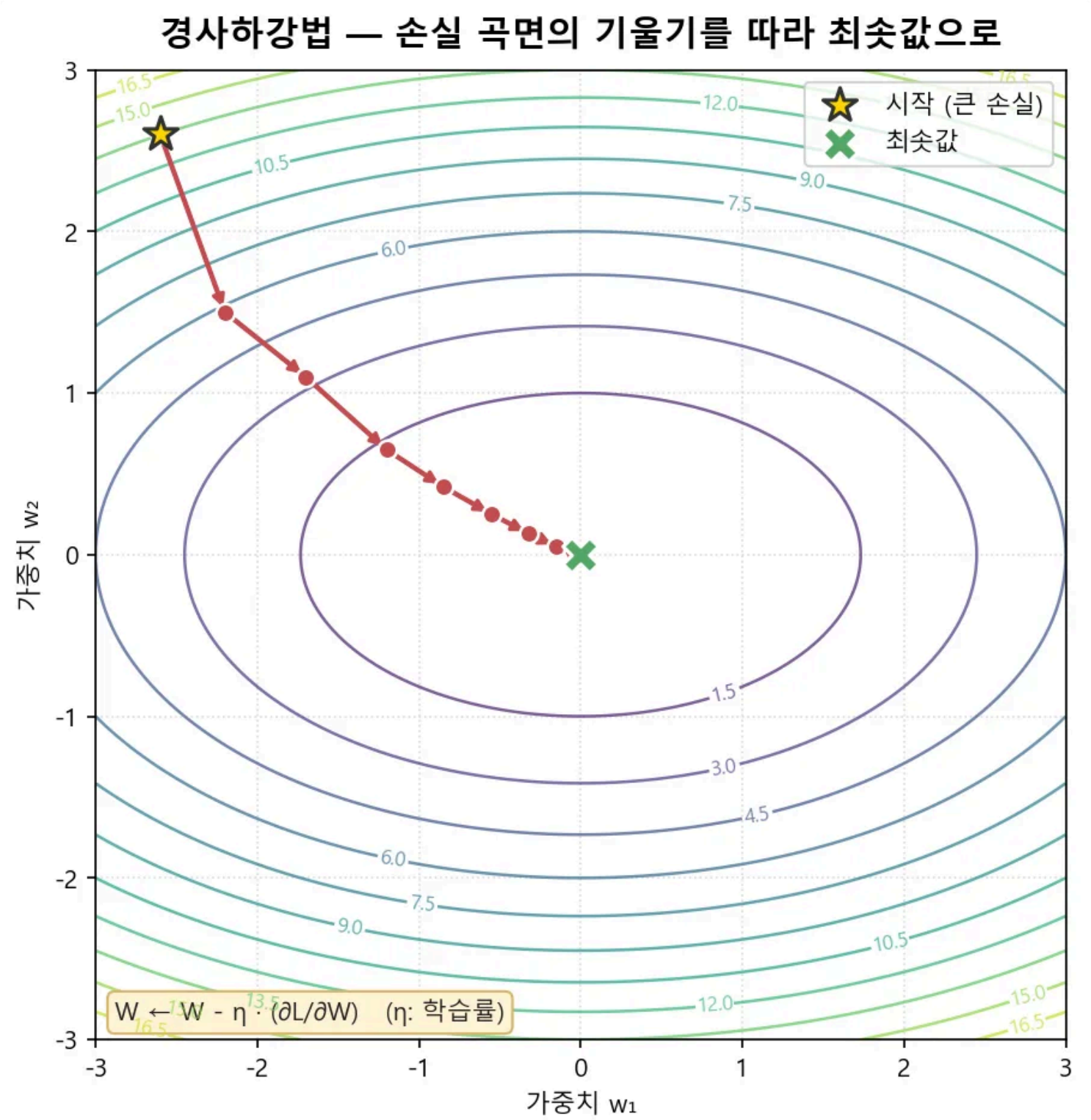
4.6 역전파 (Backpropagation)와 경사하강법

경사하강법 (Gradient Descent)

신경망의 학습은 손실함수를 최소화하는 가중치를 찾는 과정입니다. **경사하강법(Gradient Descent)**은 손실함수의 기울기(Gradient)를 따라 파라미터를 조금씩 업데이트하는 최적화 알고리즘입니다.

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

여기서 η는 **학습률(Learning Rate)**으로, 한 번에 얼마나 이동할지를 결정합니다.



> ⚠️ ***학습률의 중요성*** *- 학습률이 너무 크면: 손실값이 발산하거나 진동할 수 있습니다. - 학습률이 너무 작으면: 학습이 매우 느리고 지역 최솟값(Local Minimum)에 갇힐 수 있습니다. 일반적으로 0.001 ~ 0.01 사이의 값을 초기 학습률로 사용합니다.*

역전파 (Backpropagation)

역전파(Backpropagation)는 손실함수에 대한 각 파라미터의 기울기를 효율적으로 계산하는 알고리즘입니다. **연쇄 법칙(Chain Rule)**을 이용하여 출력층에서 입력층 방향으로 기울기를 전파합니다.

연쇄 법칙:

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial W^{(1)}}$$

계산 과정을 단계별로 정리하면 다음과 같습니다.

1. **순전파**: 입력 데이터를 통해 예측값 $\hat{y}$를 계산합니다.
2. **손실 계산**: $L = \text{Loss}(y, \hat{y})$를 구합니다.
3. **역전파**: 연쇄 법칙으로 각 파라미터의 기울기 $\frac{\partial L}{\partial W}$를 계산합니다.
4. **파라미터 업데이트**: $W \leftarrow W - \eta \frac{\partial L}{\partial W}$

```python
import numpy as np

# 간단한 1층 신경망의 역전파 예시 (MSE 손실, Sigmoid 활성화)
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(a):
    return a * (1 - a)

# 데이터
x = np.array([[1.0, 2.0]])  # 1개 샘플, 2개 특성
y = np.array([[1.0]])        # 정답

# 파라미터 초기화
np.random.seed(42)
W = np.random.randn(1, 2) * 0.1
b = np.zeros((1, 1))
lr = 0.1

# 학습 루프
for epoch in range(5):
    # 순전파
    z = x @ W.T + b
    a = sigmoid(z)

    # 손실 (MSE)
    loss = np.mean((y - a) ** 2)

    # 역전파
    dL_da = -2 * (y - a)           # 손실의 a에 대한 기울기
    da_dz = sigmoid_derivative(a)  # 활성화 함수의 기울기
    dL_dz = dL_da * da_dz

    dL_dW = dL_dz.T @ x           # W에 대한 기울기
    dL_db = np.sum(dL_dz, axis=0, keepdims=True)

    # 파라미터 업데이트
    W -= lr * dL_dW
    b -= lr * dL_db

    print(f"Epoch {epoch+1}, Loss: {loss:.6f}")
```

실제 딥러닝 프레임워크(PyTorch, TensorFlow)는 이 역전파 과정을 **자동 미분(Automatic Differentiation)** 기능으로 자동화합니다.

4.7 최적화 기법 (SGD, Adam)

확률적 경사하강법 (SGD, Stochastic Gradient Descent)

전체 데이터셋을 한 번에 사용하는 **배치 경사하강법(Batch GD)**은 데이터가 많을 때 매우 느립니다. **SGD**는 매 업데이트마다 무작위로 선택한 **하나의 샘플**만 사용하여 기울기를 계산합니다.

현실에서는 여러 샘플을 묶은 **미니배치(Mini-batch)**를 사용하는 **미니배치 SGD**가 표준입니다.

방법	업데이트에 사용하는 데이터	장점	단점
Batch GD	전체 데이터	안정적인 수렴	메모리 많이 사용, 느림
SGD	1개 샘플	빠른 업데이트	수렴이 불안정
Mini-batch SGD	N개 샘플(배치)	속도와 안정성의 균형	배치 크기 튜닝 필요

모멘텀 (Momentum)

모멘텀(Momentum)은 이전 업데이트 방향을 일부 유지하여 진동을 줄이고 수렴 속도를 높이는 기법입니다.

$$v_t = \beta v_{t-1} + (1-\beta) \nabla L$$ $$W \leftarrow W - \eta v_t$$

Adam 최적화기 (Adaptive Moment Estimation)

Adam은 현재 가장 널리 사용되는 최적화 알고리즘입니다. 모멘텀과 **RMSProp**의 장점을 결합하여 각 파라미터마다 학습률을 개별적으로 조정합니다.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L \quad \text{(1차 모멘트, 기울기 평균)}$$ $$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L)^2 \quad \text{(2차 모멘트, 기울기 분산)}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \text{(편향 보정)}$$

$$W \leftarrow W - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

기본 하이퍼파라미터는 $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$을 사용합니다.

```
import torch
import torch.nn as nn

model = nn.Linear(10, 1)

# SGD 옵티마이저
optimizer_sgd = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# Adam 옵티마이저
optimizer_adam = torch.optim.Adam(model.parameters(), lr=0.001)

# RMSProp 옵티마이저
optimizer_rmsprop = torch.optim.RMSprop(model.parameters(), lr=0.001)
```

> 💡 ***어떤 옵티마이저를 선택해야 할까?*** *초보자라면* ***Adam****을 기본값으로 사용하는 것을 권장합니다. Adam은 대부분의 경우에 잘 동작하며, 학습률에 비교적 덜 민감합니다. 학습률 10^{-3} (0.001)이 좋은 출발점입니다.*

4.8 과적합 방지 (드롭아웃, 배치 정규화, 조기 종료)

신경망은 학습 데이터에 지나치게 맞춰지는 **과적합(Overfitting)** 문제가 발생할 수 있습니다. 과적합이 발생하면 학습 데이터에서는 성능이 좋지만, 새로운 데이터(테스트 데이터)에서는 성능이 크게 떨어집니다.

드롭아웃 (Dropout)

드롭아웃(Dropout)은 학습 시 각 에포크마다 무작위로 일부 뉴런을 비활성화하는 기법입니다. 특정 뉴런에 대한 의존도를 줄여 모델이 더 강건(Robust)한 표현을 학습하도록 유도합니다.

드롭아웃 비율 p를 0.5로 설정하면 학습 시 절반의 뉴런이 무작위로 꺼집니다. 테스트 시에는 모든 뉴런을 활성화하고 출력에 $(1-p)$를 곱하여 스케일을 조정합니다.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(p=0.5),     # 50% 뉴런 비활성화
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(p=0.3),     # 30% 뉴런 비활성화
    nn.Linear(128, 10),
    nn.Softmax(dim=1)
)
```

배치 정규화 (Batch Normalization)

배치 정규화(Batch Normalization)는 각 층의 입력을 미니배치 단위로 정규화(평균 0, 분산 1)하는 기법입니다. 학습을 안정시키고, 학습 속도를 높이며, 과적합도 줄이는 효과가 있습니다.

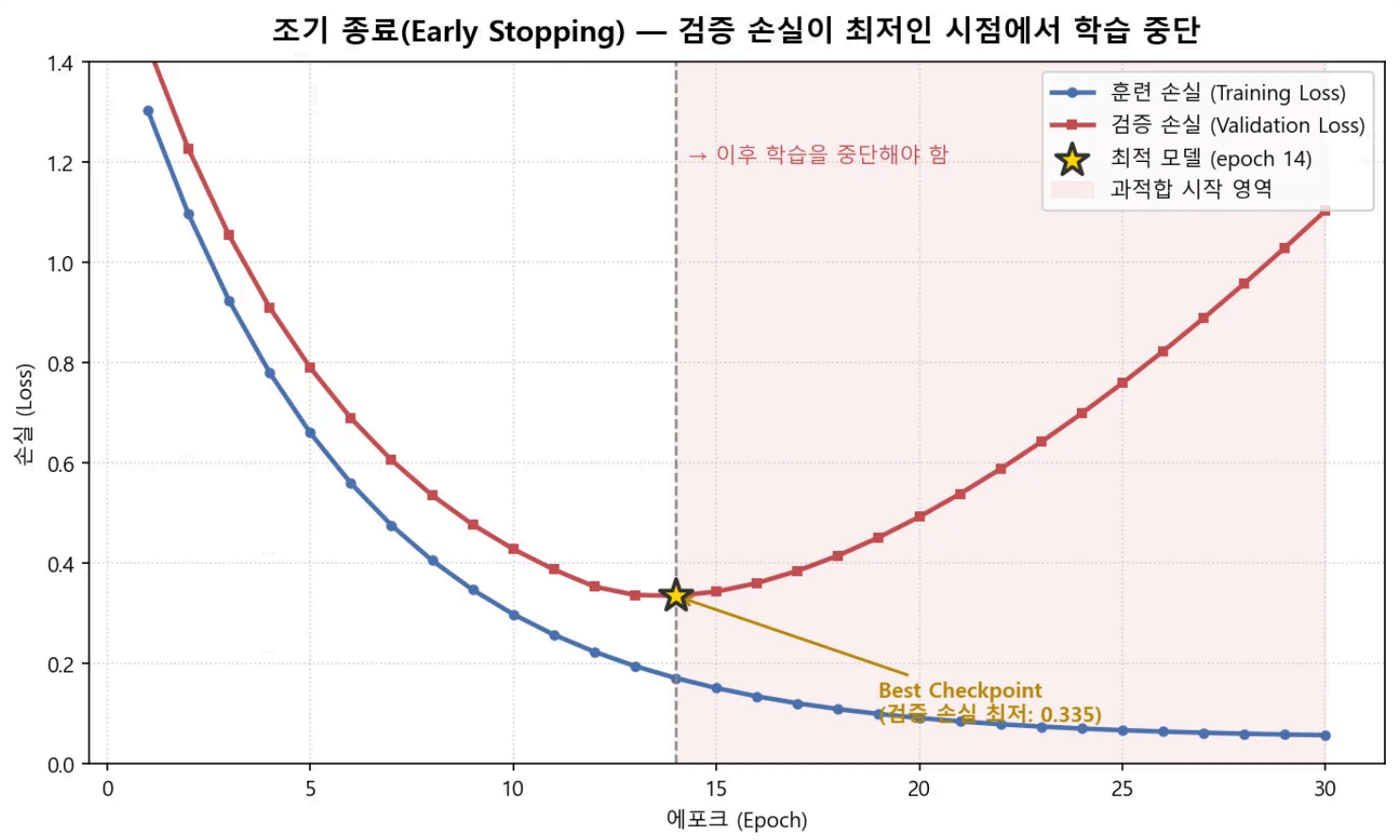
$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$ $$y_i = \gamma \hat{x}_i + \beta$$

여기서 μ_B와 σ_B^2는 미니배치의 평균과 분산, γ와 β는 학습 가능한 스케일·시프트 파라미터입니다.

```
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.BatchNorm1d(256),   # 배치 정규화
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
```

조기 종료 (Early Stopping)

조기 종료(Early Stopping)는 학습 과정에서 검증 손실(Validation Loss)이 더 이상 줄어들지 않으면 학습을 멈추는 기법입니다.




```
# 조기 종료 구현 예시
best_val_loss = float('inf')
patience = 5      # 개선 없이 허용하는 에포크 수
counter = 0

for epoch in range(100):
    train_loss = train_one_epoch(model, train_loader)
    val_loss = evaluate(model, val_loader)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), 'best_model.pth')
        counter = 0
    else:
        counter += 1
        if counter ≥ patience:
            print(f"Early Stopping at epoch {epoch+1}")
            break

# 최적 모델 복원
model.load_state_dict(torch.load('best_model.pth'))
```

> 📌 ***정규화 기법 요약*** *| 기법 | 원리 | 효과 | |---|---|---| | 드롭아웃 | 뉴런 무작위 비활성화 | 과적합 방지 | | 배치 정규화 | 층 입력 정규화 | 학습 안정화, 속도 향상 | | 조기 종료 | 검증 손실 모니터링 | 과적합 방지 | | L1/L2 정규화 | 손실함수에 가중치 패널티 추가 | 가중치 희소화/축소 |*

[실습] 간단한 신경망으로 손글씨 숫자 분류 (MNIST)

이제 지금까지 배운 개념을 모두 활용하여 **MNIST** 데이터셋으로 손글씨 숫자(0~9)를 분류하는 신경망을 구현해보겠습니다.

> 💡 ***MNIST 데이터셋*** *MNIST(Modified National Institute of Standards and Technology)는 0~9 손글씨 숫자 이미지 70,000장(훈련 60,000장 + 테스트 10,000장)으로 구성된 데이터셋입니다. 각 이미지는 28×28 픽셀의 흑백 이미지입니다. 딥러닝의 "Hello World"로 불릴 만큼 입문용으로 널리 사용됩니다.*

전체 구현 코드

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

# ──────────────────────────────────────────
# 1. 하이퍼파라미터 설정
# ──────────────────────────────────────────
BATCH_SIZE = 64
LEARNING_RATE = 0.001
EPOCHS = 10
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

print(f"사용 디바이스: {DEVICE}")

# ──────────────────────────────────────────
# 2. 데이터 로드 및 전처리
# ──────────────────────────────────────────
transform = transforms.Compose([
    transforms.ToTensor(),                  # PIL 이미지 → Tensor [0,1]
    transforms.Normalize((0.1307,), (0.3081,))  # MNIST 평균/표준편차로 정규화
])

train_dataset = datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)
test_dataset = datasets.MNIST(
    root='./data', train=False, download=True, transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader  = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print(f"훈련 샘플 수: {len(train_dataset)}, 테스트 샘플 수: {len(test_dataset)}")

# ──────────────────────────────────────────
# 3. 데이터 시각화
# ──────────────────────────────────────────
```

```
examples = iter(train_loader)
images, labels = next(examples)

fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(images[i].squeeze(), cmap='gray')
    ax.set_title(f'Label: {labels[i].item()}')
    ax.axis('off')
plt.suptitle('MNIST 샘플 이미지')
plt.tight_layout()
plt.show()

# ──────────────────────────────────────────
# 4. 신경망 모델 정의
# ──────────────────────────────────────────
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.network = nn.Sequential(
            nn.Flatten(),                    # (B, 1, 28, 28) → (B, 784)

            nn.Linear(784, 256),             # 입력층 → 은닉층 1
            nn.BatchNorm1d(256),             # 배치 정규화
            nn.ReLU(),
            nn.Dropout(p=0.3),              # 드롭아웃

            nn.Linear(256, 128),             # 은닉층 1 → 은닉층 2
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(p=0.3),

            nn.Linear(128, 10)               # 은닉층 2 → 출력층 (10 클래스)
            # CrossEntropyLoss에 Softmax가 내장되어 있으므로 별도 Softmax 불필요
        )

    def forward(self, x):
        return self.network(x)

model = MLP().to(DEVICE)
print(model)
print(f"\n총 파라미터 수: {sum(p.numel() for p in model.parameters()):,}")

# ──────────────────────────────────────────
```

```
# 5. 손실함수와 옵티마이저 정의
# ──────────────────────────────────────────
criterion = nn.CrossEntropyLoss()          # 다중 분류 → 교차 엔트로피
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

# ──────────────────────────────────────────
# 6. 학습 함수
# ──────────────────────────────────────────
def train(model, loader, criterion, optimizer, device):
    model.train()   # 학습 모드 (Dropout, BN 활성화)
    total_loss, correct = 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()           # 기울기 초기화
        outputs = model(images)         # 순전파
        loss = criterion(outputs, labels)  # 손실 계산
        loss.backward()                 # 역전파
        optimizer.step()                # 파라미터 업데이트

        total_loss += loss.item()
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()

    avg_loss = total_loss / len(loader)
    accuracy = 100. * correct / len(loader.dataset)
    return avg_loss, accuracy

# ──────────────────────────────────────────
# 7. 평가 함수
# ──────────────────────────────────────────
def evaluate(model, loader, criterion, device):
    model.eval()    # 평가 모드 (Dropout 비활성화, BN 추론 통계 사용)
    total_loss, correct = 0, 0

    with torch.no_grad():               # 기울기 계산 비활성화
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item()
```

```
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()

    avg_loss = total_loss / len(loader)
    accuracy = 100. * correct / len(loader.dataset)
    return avg_loss, accuracy

# ──────────────────────────────────────────
# 8. 학습 실행 (조기 종료 포함)
# ──────────────────────────────────────────
train_losses, val_losses = [], []
train_accs, val_accs = [], []

best_val_loss = float('inf')
patience, counter = 5, 0

for epoch in range(1, EPOCHS + 1):
    train_loss, train_acc = train(model, train_loader, criterion, optimizer, DEVICE)
    val_loss, val_acc = evaluate(model, test_loader, criterion, DEVICE)

    train_losses.append(train_loss)
    val_losses.append(val_loss)
    train_accs.append(train_acc)
    val_accs.append(val_acc)

    print(f"Epoch [{epoch:2d}/{EPOCHS}] "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}% | "
          f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%")

    # 조기 종료
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), 'best_mnist_model.pth')
        counter = 0
    else:
        counter += 1
        if counter >= patience:
            print(f"\n조기 종료: {epoch}번째 에포크에서 종료합니다.")
            break

# 최적 모델 복원
model.load_state_dict(torch.load('best_mnist_model.pth'))
print(f"\n최적 모델 검증 손실: {best_val_loss:.4f}")
```

```
# ──────────────────────────────────────────
# 9. 학습 결과 시각화
# ──────────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(train_losses, label='Train Loss')
axes[0].plot(val_losses, label='Validation Loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('학습 손실 곡선')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(train_accs, label='Train Accuracy')
axes[1].plot(val_accs, label='Validation Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy (%)')
axes[1].set_title('정확도 곡선')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()

# ──────────────────────────────────────────
# 10. 예측 결과 시각화
# ──────────────────────────────────────────
model.eval()
test_images, test_labels = next(iter(test_loader))
test_images = test_images.to(DEVICE)

with torch.no_grad():
    outputs = model(test_images)
    _, predictions = outputs.max(1)

test_images = test_images.cpu()
predictions = predictions.cpu()

fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(test_images[i].squeeze(), cmap='gray')
    color = 'green' if predictions[i] == test_labels[i] else 'red'
```

```
    ax.set_title(f'예측: {predictions[i].item()} / 정답: {test_labels[i].item()}',
                 color=color)
    ax.axis('off')
plt.suptitle('예측 결과 (초록: 정답, 빨강: 오답)')
plt.tight_layout()
plt.show()
```

실행 결과 예시

위 코드를 실행하면 약 10 에포크 이내에 다음과 같은 결과를 얻을 수 있습니다.

```
Epoch [ 1/10] Train Loss: 0.2847, Train Acc: 91.45% | Val Loss: 0.1123, Val Acc: 96.58%
Epoch [ 2/10] Train Loss: 0.1421, Train Acc: 95.73% | Val Loss: 0.0891, Val Acc: 97.21%
 ...
Epoch [10/10] Train Loss: 0.0612, Train Acc: 98.12% | Val Loss: 0.0712, Val Acc: 97.95%
```

> 💡 ***결과 해석*** *약 98% 수준의 테스트 정확도를 달성합니다. 이는 100장 중 약 98장의 손글씨 숫자를 올바르게 분류한다는 의미입니다. 이후 챕터에서 배울 합성곱 신경망(CNN)을 사용하면 99% 이상의 정확도도 달성할 수 있습니다.*

마치며

이번 챕터에서는 현대 딥러닝의 핵심 기초가 되는 신경망의 모든 구성 요소를 살펴보았습니다. 생물학적 뉴런에서 출발하여, 퍼셉트론, 다층 퍼셉트론, 활성화 함수, 순전파, 손실함수, 역전파, 최적화, 과적합 방지까지 신경망의 학습 파이프라인 전체를 이해했습니다.

> 📌 ***핵심 요약*** *| 개념 | 역할 | |---|---| |* ***인공 뉴런*** *| 가중합 + 활성화 함수로 신호 처리 | |* ***다층 퍼셉트론(MLP)*** *| 은닉층을 쌓아 비선형 패턴 학습 | |* ***활성화 함수*** *| ReLU(은닉층), Sigmoid(이진), Softmax(다중 분류) | |* ***순전파*** *| 입력 → 출력 방향으로 예측값 계산 | |* ***손실함수*** *| MSE(회귀), Cross-Entropy(분류) | |* ***역전파*** *| 연쇄 법칙으로 기울기 계산 | |* ***경사하강법*** *| 기울기를 따라 파라미터 업데이트 | |* ***Adam*** *| 학습률 자동 조정 최적화기 | |* ***드롭아웃/배치 정규화*** *| 과적합 방지 및 학습 안정화 |*

다음 챕터 예고: Chapter 5. 합성곱 신경망 (CNN)

Chapter 5에서는 이미지 처리에 특화된 **합성곱 신경망(CNN, Convolutional Neural Network)**을 다룹니다. CNN은 MLP와 달리 이미지의 공간적 구조(위치, 인접 픽셀 관계)를 직접 활용하여 훨씬 뛰어난 이미지 인식 성능을 달성합니다. 합성곱 층, 풀링 층, 유명 CNN 아키텍처(LeNet, VGG, ResNet)를 배우고 CIFAR-10 이미지 분류를 직접 구현해봅니다.

우리는 앞선 챕터에서 신경망의 기본 구조와 학습 원리를 배웠습니다. 그런데 신경망이 이미지를 처리할 때는 특별한 문제가 생깁니다. 가령 28×28 픽셀짜리 흑백 이미지를 완전연결층(Fully Connected Layer)으로 처리하려면 784개의 입력 뉴런이 필요하고, 고해상도 컬러 이미지라면 수백만 개의 파라미터가 필요해집니다. 이처럼 기존 방식은 이미지의 공간적 구조를 전혀 활용하지 못하고, 계산 비용도 폭발적으로 증가합니다.

합성곱 신경망(CNN, Convolutional Neural Network)은 이 문제를 해결하기 위해 고안된 구조입니다. CNN은 이미지의 지역적 패턴을 효율적으로 학습하여 이미지 분류, 객체 탐지, 얼굴 인식 등 다양한 시각 지능 과제에서 놀라운 성능을 발휘합니다.

> 💡 ***학습 목표***
>
> - *이미지 데이터가 어떻게 표현되는지 이해합니다.*
> - *합성곱 연산의 원리를 이해합니다.*
> - *필터, 스트라이드, 패딩의 역할을 설명할 수 있습니다.*
> - *풀링(Pooling)과 특성맵(Feature Map)의 개념을 이해합니다.*
> - *CNN의 전체 구조를 파악합니다.*
> - *LeNet부터 ResNet까지 대표 모델의 발전 흐름을 이해합니다.*
> - *CNN의 다양한 활용 분야를 파악합니다.*
> - *실습을 통해 CNN으로 이미지 분류를 직접 체험합니다.*

5.1 이미지 데이터의 특성과 표현

이미지는 숫자의 배열입니다

컴퓨터에서 이미지는 **픽셀(Pixel)**의 격자로 표현됩니다. 각 픽셀은 밝기나 색상 정보를 담은 숫자입니다.

- **흑백(Grayscale) 이미지**: 각 픽셀이 0(검정)~255(흰색) 사이의 숫자 한 개로 표현됩니다. 28×28 이미지는 2차원 배열 (28, 28)로 나타냅니다.
- **컬러(RGB) 이미지**: 각 픽셀이 빨강(R), 초록(G), 파랑(B) 세 채널의 값으로 구성됩니다. 28×28 컬러 이미지는 3차원 배열 (28, 28, 3)으로 나타냅니다.

```
import numpy as np
from PIL import Image

# 28×28 흑백 이미지를 numpy 배열로 표현
gray_image = np.random.randint(0, 256, (28, 28), dtype=np.uint8)
print("흑백 이미지 shape:", gray_image.shape)   # (28, 28)

# 32×32 컬러 이미지를 numpy 배열로 표현
color_image = np.random.randint(0, 256, (32, 32, 3), dtype=np.uint8)
print("컬러 이미지 shape:", color_image.shape)  # (32, 32, 3)
```

딥러닝 프레임워크에서는 보통 배치(Batch) 차원을 포함하여 **(배치 크기, 높이, 너비, 채널)** 또는 **(배치 크기, 채널, 높이, 너비)** 형태로 데이터를 다룹니다.

이미지 데이터의 공간적 특성

이미지는 단순한 숫자의 나열이 아닙니다. 이미지에는 다음과 같은 중요한 공간적 특성이 있습니다.

- **지역 상관성(Local Correlation)**: 인접한 픽셀들은 서로 연관된 값을 가집니다. 강아지의 눈 주변 픽셀들은 서로 비슷한 패턴을 형성합니다.
- **계층적 구조(Hierarchical Structure)**: 낮은 수준에서는 선(edge)이나 색상 같은 단순한 패턴이, 높은 수준에서는 귀, 눈 같은 복잡한 특징이 나타납니다.
- **위치 불변성(Translation Invariance)**: 고양이가 사진 왼쪽에 있든 오른쪽에 있든 "고양이"로 인식해야 합니다.

> 📌 *CNN은 바로 이러한 이미지의 공간적 특성을 적극 활용하도록 설계된 구조입니다. 완전연결층은 픽셀을 일렬로 펼쳐서 처리하기 때문에 이 공간 정보를 잃어버리지만, CNN은 공간 구조를 유지한 채로 학습합니다.*

5.2 합성곱(Convolution) 연산의 원리

합성곱이란 무엇인가

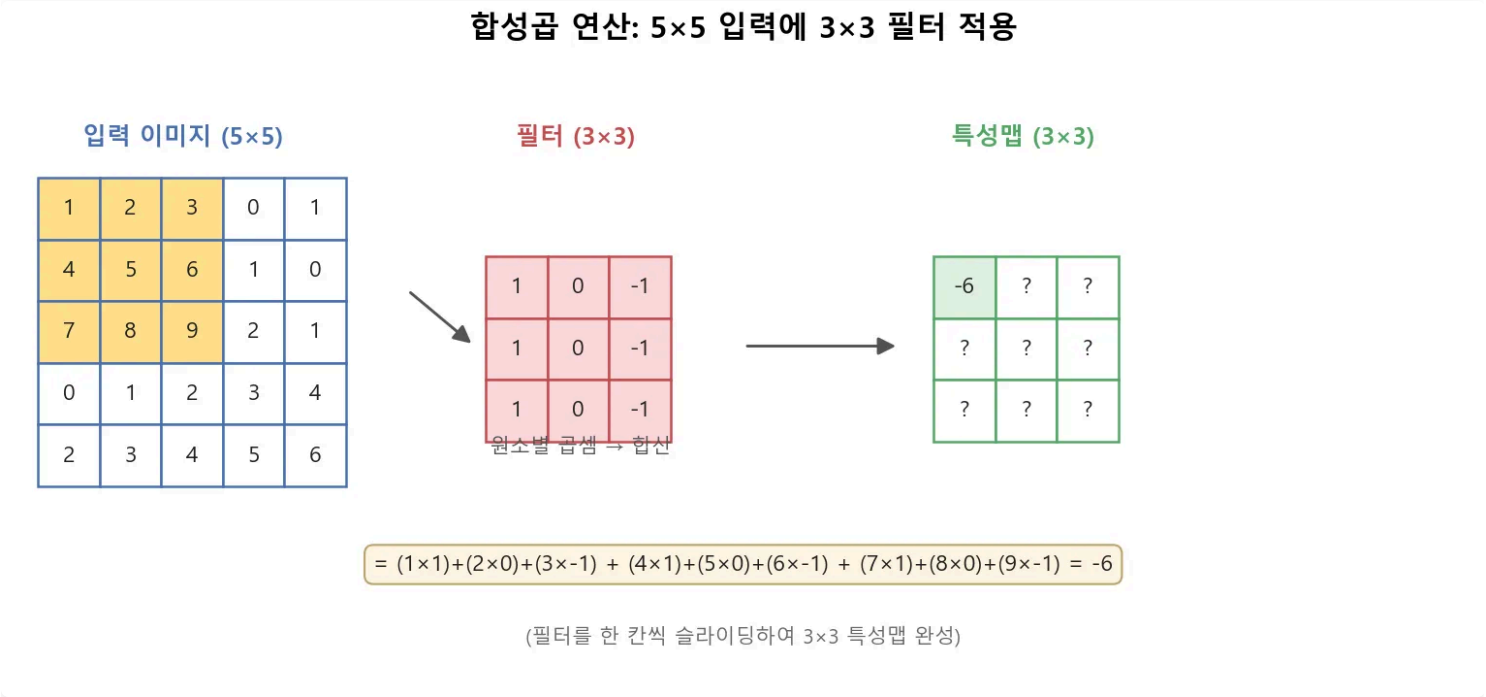
합성곱(Convolution)은 이미지 위를 작은 창(필터)이 이동하면서 지역적인 특징을 추출하는 연산입니다. 신호 처리나 이미지 처리에서 오래전부터 사용되던 수학적 연산을 딥러닝에 적용한 것입니다.

직관적으로 설명하면 다음과 같습니다.

1. 작은 창(**필터** 또는 **커널**)을 이미지 위에 올려놓습니다.
2. 창이 덮는 영역의 픽셀값과 필터값을 각각 곱한 뒤 모두 더합니다.
3. 그 결과값 하나를 새로운 위치에 기록합니다.
4. 창을 옆으로 이동하면서 이 과정을 반복합니다.

합성곱 연산 예시

예를 들어, 5×5 이미지에 3×3 필터를 적용하는 경우를 살펴봅시다.



이렇게 필터를 이미지 전체에 걸쳐 슬라이딩하면 **특성맵(Feature Map)**이 생성됩니다.

```
import numpy as np

def simple_convolution(image, kernel):
    """간단한 2D 합성곱 연산 구현"""
    i_h, i_w = image.shape
    k_h, k_w = kernel.shape
    out_h = i_h - k_h + 1
    out_w = i_w - k_w + 1
    output = np.zeros((out_h, out_w))

    for i in range(out_h):
        for j in range(out_w):
            output[i, j] = np.sum(image[i:i+k_h, j:j+k_w] * kernel)

    return output

# 예시
image = np.array([
    [1, 2, 3, 0, 1],
    [4, 5, 6, 1, 0],
    [7, 8, 9, 2, 1],
    [0, 1, 2, 3, 4],
    [2, 3, 4, 5, 6]
], dtype=float)

# 세로 엣지를 감지하는 필터
kernel = np.array([
    [ 1,  0, -1],
    [ 1,  0, -1],
    [ 1,  0, -1]
], dtype=float)

result = simple_convolution(image, kernel)
print("특성맵:")
print(result)
```

필터가 학습하는 것

CNN에서 필터의 값들은 사람이 직접 정하지 않고, **역전파(Backpropagation)**를 통해 자동으로 학습됩니다. 학습 과정에서 CNN은 스스로 유용한 특징을 추출하는 필터를 찾아냅니다.

초기 층에서는 단순한 엣지나 색상을 감지하는 필터가 학습되고, 깊은 층으로 갈수록 눈, 코, 귀처럼 복잡한 패턴을 감지하는 필터가 학습됩니다.

> 💡 *완전연결층은 각 뉴런이 모든 입력과 연결되므로 파라미터 수가 매우 많습니다. 반면 합성곱층은 필터의 가중치만 학습하면 되므로 파라미터 수가 훨씬 적고, 같은 필터를 이미지 전체에 재사용하는 **가중치 공유(Weight Sharing)** 덕분에 효율적입니다.*

5.3 필터, 스트라이드, 패딩

필터(Filter) / 커널(Kernel)

필터(Filter)는 합성곱 연산에서 이미지를 슬라이딩하는 작은 행렬입니다. **커널(Kernel)**이라고도 부릅니다.

- 필터의 크기는 보통 3×3, 5×5, 7×7 등 홀수를 사용합니다.
- 하나의 합성곱층은 여러 개의 필터를 사용하여 다양한 특징을 동시에 추출합니다.
- 필터 수가 많을수록 더 다양한 특성맵이 생성되어 표현력이 높아집니다.

스트라이드(Stride)

스트라이드(Stride)는 필터가 한 번에 이동하는 칸 수입니다.

- **스트라이드 1**: 필터를 한 칸씩 이동 → 출력 크기가 입력과 비슷하게 유지됩니다.
- **스트라이드 2**: 필터를 두 칸씩 이동 → 출력 크기가 절반으로 줄어듭니다.

출력 크기를 계산하는 공식은 다음과 같습니다.

```
출력 크기 = (입력 크기 - 필터 크기) / 스트라이드 + 1
```

예시: 입력 7×7, 필터 3×3, 스트라이드 1 → (7-3)/1 + 1 = 5 → 출력 5×5

입력 크기	필터 크기	스트라이드	출력 크기
7×7	3×3	1	5×5
7×7	3×3	2	3×3
28×28	5×5	1	24×24
32×32	3×3	1	30×30

패딩(Padding)

합성곱 연산을 반복하면 출력 크기가 점점 작아집니다. **패딩(Padding)**은 이를 방지하기 위해 입력 이미지 가장자리에 0 값을 채워 넣는 기법입니다.

- **제로 패딩(Zero Padding)**: 가장자리에 0을 채워 넣습니다.
- **'valid' 패딩**: 패딩 없이 연산 → 출력 크기가 줄어듭니다.
- **'same' 패딩**: 출력 크기를 입력과 같게 유지하도록 패딩을 자동 추가합니다.

```
import tensorflow as tf
from tensorflow.keras import layers

# 패딩 비교 예시
model_valid = layers.Conv2D(32, (3, 3), padding='valid', input_shape=(28, 28, 1))
model_same  = layers.Conv2D(32, (3, 3), padding='same',  input_shape=(28, 28, 1))

# valid: 입력 28x28 → 출력 26x26
# same:  입력 28x28 → 출력 28x28 (크기 유지)
```

패딩을 사용하면 이미지 가장자리 픽셀도 충분히 합성곱에 참여할 수 있어 정보 손실을 줄일 수 있습니다.

> ⚠️ *스트라이드와 패딩은 출력 크기와 계산량에 직접 영향을 줍니다. 모델 설계 시 이 두 값을 적절히 조합하여 원하는 출력 크기를 맞추는 것이 중요합니다.*

5.4 풀링(Pooling)과 특성맵(Feature Map)

특성맵(Feature Map)

특성맵(Feature Map)은 합성곱 연산의 결과로 생성되는 2차원 배열입니다. 하나의 필터를 적용하면 하나의 특성맵이 만들어집니다. 여러 개의 필터를 사용하면 그 수만큼 특성맵이 쌓입니다.

특성맵의 각 값은 해당 위치에서 필터가 감지하는 특징이 얼마나 강하게 나타났는지를 의미합니다. 값이 클수록 그 위치에서 해당 패턴이 강하게 검출된 것입니다.

풀링(Pooling)이란

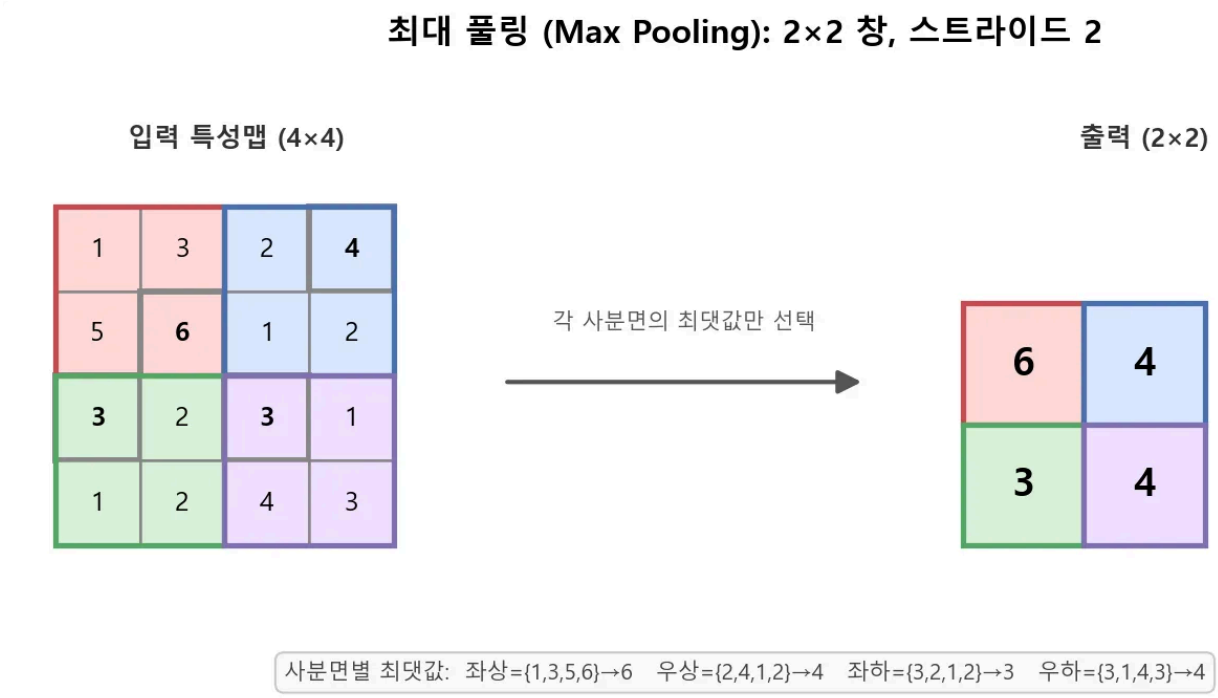
풀링(Pooling)은 특성맵의 크기를 줄이면서 중요한 정보를 유지하는 연산입니다. 다운샘플링(Downsampling)이라고도 합니다.

풀링의 주요 역할은 다음과 같습니다.

- 특성맵의 크기를 줄여 계산량을 감소시킵니다.
- 위치의 소소한 변화에 강건하게(robust) 만들어 줍니다.
- 과적합(Overfitting)을 억제하는 데 도움이 됩니다.

최대 풀링(Max Pooling)

가장 많이 사용되는 풀링 방식입니다. 풀링 창 내에서 가장 큰 값을 선택합니다.



각 사분면의 최댓값만 선택되어 출력 2×2가 만들어집니다. 좌상 6, 우상 4, 좌하 3, 우하 4.

평균 풀링(Average Pooling)

창 내 값들의 평균을 계산합니다. 최대 풀링보다 부드러운 특성맵을 생성하지만, 일반적으로 이미지 분류에서는 최대 풀링이 더 좋은 성능을 보입니다.

구분	최대 풀링 (Max Pooling)	평균 풀링 (Average Pooling)
연산	창 내 최댓값 선택	창 내 평균값 계산
특징	가장 두드러진 특징 추출	전체적인 특징 반영
사용	이미지 분류에 주로 사용	GAP 등 특수 목적에 활용

```
import tensorflow as tf
from tensorflow.keras import layers

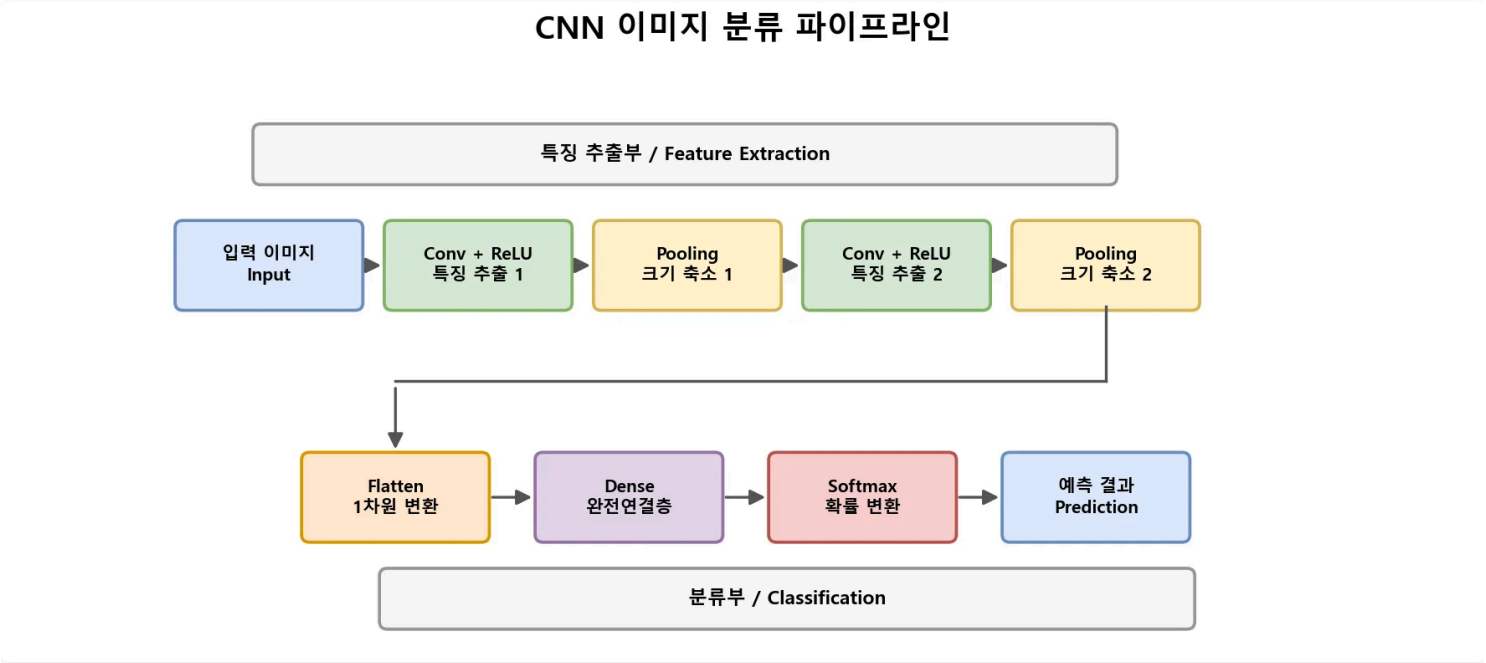
# 최대 풀링: 2x2 창, 스트라이드 2
max_pool = layers.MaxPooling2D(pool_size=(2, 2), strides=2)

# 평균 풀링: 2x2 창, 스트라이드 2
avg_pool = layers.AveragePooling2D(pool_size=(2, 2), strides=2)
```

> 📌 *최대 풀링은 가장 강하게 반응한 특징만 남기기 때문에, 이미지의 작은 이동이나 왜곡에도 안정적인 결과를 만들어 냅니다. 이를 **위치 불변성(Translation Invariance)**이라고 합니다.*

5.5 CNN의 전체 구조

CNN은 크게 **특징 추출부**와 **분류부**로 나뉩니다.



각 층의 역할

합성곱층(Convolutional Layer)은 필터를 사용해 입력에서 지역적인 특징을 추출합니다. 층이 깊어질수록 더 복잡하고 추상적인 특징을 학습합니다.

활성화 함수(Activation Function)는 보통 **ReLU(Rectified Linear Unit)**를 사용합니다. 음수를 0으로 만들어 비선형성을 부여합니다.

풀링층(Pooling Layer)은 특성맵의 크기를 줄여 계산 효율을 높이고 과적합을 억제합니다.

플래튼(Flatten)은 2D 특성맵을 1D 벡터로 변환하여 완전연결층에 전달합니다.

완전연결층(Fully Connected Layer)은 추출된 특징들을 종합하여 최종 분류를 수행합니다.

간단한 CNN 예시 코드

```
import tensorflow as tf
from tensorflow.keras import layers, models

def build_simple_cnn(input_shape=(28, 28, 1), num_classes=10):
    model = models.Sequential([
        # 첫 번째 합성곱 블록
        layers.Conv2D(32, (3, 3), activation='relu', padding='same',
                      input_shape=input_shape),
        layers.MaxPooling2D((2, 2)),

        # 두 번째 합성곱 블록
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),

        # 분류부
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])
    return model

model = build_simple_cnn()
model.summary()
```

이 모델에서 데이터 흐름을 따라가 보면 다음과 같습니다.

층	출력 Shape	파라미터 수
Conv2D (32필터, 3×3)	(26, 26, 32)	320
MaxPooling2D (2×2)	(13, 13, 32)	0
Conv2D (64필터, 3×3)	(11, 11, 64)	18,496
MaxPooling2D (2×2)	(5, 5, 64)	0
Flatten	(1600,)	0
Dense (128)	(128,)	204,928
Dense (10)	(10,)	1,290

> 💡 *합성곱층은 파라미터 수가 매우 적지만, 그 뒤의 완전연결층에서 파라미터가 급격히 늘어납니다. 현대 CNN은 완전연결층 이전에 **전역 평균 풀링(Global Average Pooling)**을 사용하여 파라미터 수를 줄이는 경향이 있습니다.*

5.6 대표 모델의 발전 - LeNet → AlexNet → VGG → ResNet

CNN은 수십 년에 걸쳐 꾸준히 발전해왔습니다. 핵심 모델들의 등장과 의의를 살펴봅시다.

LeNet-5 (1998) — CNN의 원조

얀 르쿤(Yann LeCun)이 설계한 LeNet-5는 최초의 실용적인 CNN 모델입니다. 손으로 쓴 숫자(MNIST)를 인식하는 데 사용되었습니다.

- **구조**: Conv → Pool → Conv → Pool → FC → FC → 출력
- **의의**: 합성곱과 풀링을 결합하는 CNN의 기본 패턴을 확립하였습니다.
- **한계**: 당시 컴퓨팅 파워 부족으로 더 깊은 모델을 만들기 어려웠습니다.

AlexNet (2012) — 딥러닝 혁명의 시작

2012년 ImageNet 대회에서 AlexNet은 기존 방법보다 오류율을 10% 이상 낮추며 딥러닝 시대를 열었습니다.

- **구조**: 5개의 합성곱층 + 3개의 완전연결층, 총 8층
- **혁신 포인트**:
- **ReLU 활성화 함수** 사용 (Sigmoid 대비 학습 속도 대폭 향상)
- **드롭아웃(Dropout)** 도입으로 과적합 방지
- **GPU** 2개를 병렬로 사용하는 학습
- **데이터 증강(Data Augmentation)** 적용
- **의의**: CNN이 실제로 깊고 강력한 모델이 될 수 있음을 증명하였습니다.

VGGNet (2014) — 단순함의 미학

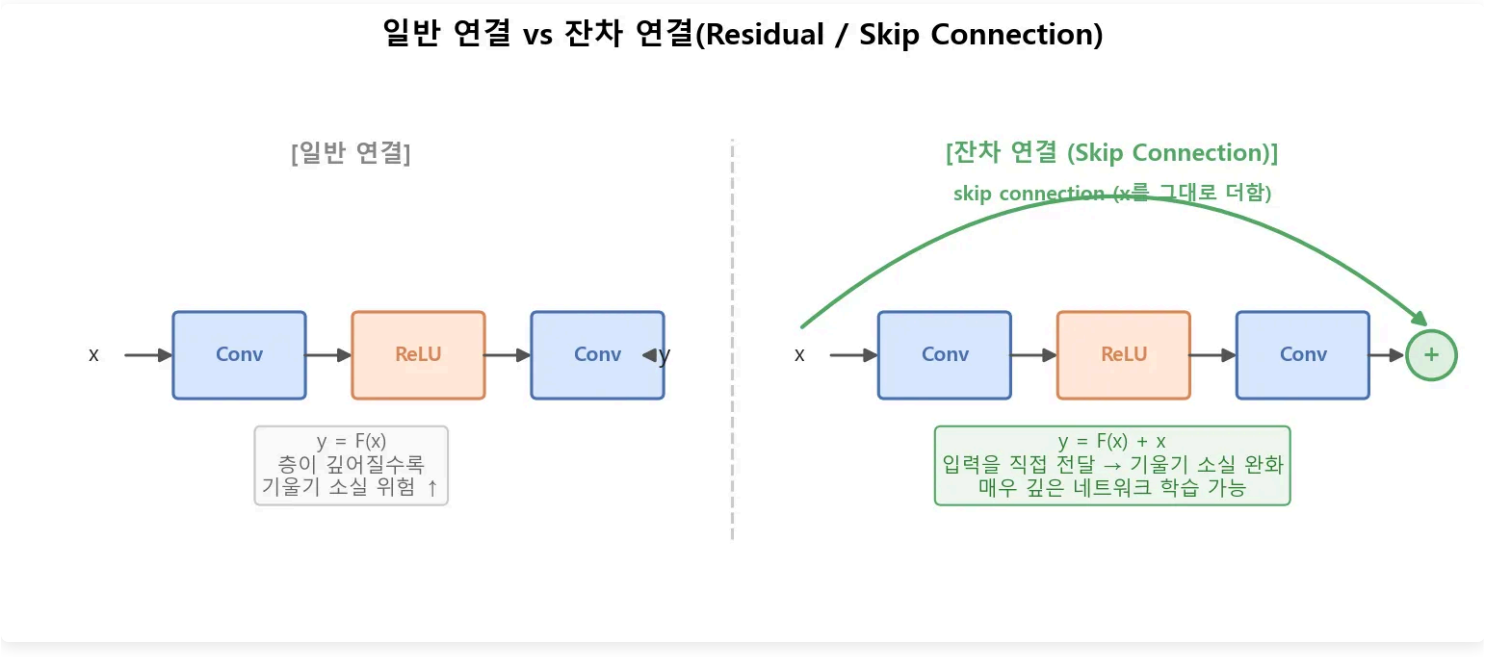
옥스퍼드 대학팀이 개발한 VGGNet은 단순하지만 매우 깊은 구조로 높은 성능을 달성하였습니다.

- **구조**: 3×3 작은 필터만 일관되게 사용, 16~19층
- **혁신 포인트**:
- 3×3 필터 두 번 = 5×5 필터 한 번과 같은 수용 영역(receptive field)이지만 파라미터 수가 적음
- 단순하고 균일한 구조로 재현성이 높음
- **한계**: 파라미터 수가 약 1억 4천만 개로 매우 많아 메모리 부담이 큽니다.

ResNet (2015) — 깊이의 한계를 극복

마이크로소프트 리서치의 허카이밍(Kaiming He) 팀이 개발한 ResNet은 **잔차 연결(Residual Connection)** 또는 **스킵 연결(Skip Connection)**을 도입하였습니다.

- **구조**: 152층(ResNet-152)까지 구현
- **핵심 아이디어**: 잔차 블록(Residual Block)



입력 x가 출력에 그대로 더해지므로, 층이 깊어져도 기울기 소실(Gradient Vanishing) 문제 없이 학습이 가능합니다.

- **의의**: 매우 깊은 네트워크를 안정적으로 학습할 수 있게 되어, 층 수와 성능이 비례하는 시대를 열었습니다.

모델 비교 요약

모델	연도	층 수	특징	Top-5 오류율
LeNet-5	1998	7층	CNN의 기반 확립	(MNIST 기준)
AlexNet	2012	8층	ReLU, Dropout, GPU 학습	15.3%
VGGNet-16	2014	16층	3×3 필터 일관 사용	7.3%
ResNet-50	2015	50층	잔차 연결, 초심층 학습	5.25%

> 📌 *ResNet 이후에도 DenseNet, EfficientNet, Vision Transformer(ViT) 등 다양한 모델이 등장하였습니다. 그러나 이 모델들 모두 CNN의 핵심 원리를 바탕으로 발전한 것입니다.*

5.7 CNN의 활용

CNN은 이미지와 관련된 거의 모든 분야에서 활용됩니다.

이미지 분류(Image Classification)

입력 이미지가 어떤 클래스에 속하는지 판단합니다.

- **의료 진단**: X-ray, MRI 이미지에서 폐암, 당뇨망막병증 등을 감지합니다.
- **제품 품질 검사**: 공장 생산라인에서 불량품을 자동 감지합니다.
- **식물 병해 감지**: 농작물 잎 사진으로 병충해를 진단합니다.

객체 탐지(Object Detection)

이미지에서 여러 객체의 위치(Bounding Box)와 종류를 동시에 찾아냅니다.

- **자율주행**: 도로 위 차량, 보행자, 신호등을 실시간 감지합니다.
- **CCTV 분석**: 특정 인물이나 물체를 자동으로 추적합니다.
- **대표 모델**: YOLO, Faster R-CNN, SSD 등

얼굴 인식(Face Recognition)

얼굴을 감지하고, 누구인지 식별하거나 감정을 분석합니다.

- **스마트폰 잠금 해제**: Face ID 같은 생체 인증 시스템
- **출입 통제 시스템**: 등록된 직원 얼굴을 인식하여 출입 허용
- **감정 분석**: 표정에서 기쁨, 슬픔, 분노 등 감정 파악

이미지 생성 및 변환

- **스타일 트랜스퍼(Style Transfer)**: 사진을 특정 화풍의 그림처럼 변환합니다.
- **이미지 초해상도(Super-Resolution)**: 저화질 이미지를 고화질로 변환합니다.
- **의료 이미지 분할(Segmentation)**: 종양의 경계를 픽셀 단위로 정확히 구분합니다.

> 💡 *CNN은 이미지 처리를 넘어 동영상 분석, 자연어 처리, 음성 인식 등 다양한 분야에도 응용되고 있습니다.*

[실습] CNN으로 이미지 분류 체험

MNIST 손글씨 숫자 데이터셋을 이용해 CNN 모델을 만들고 학습시켜 봅시다.

1단계: 데이터 준비

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# MNIST 데이터셋 불러오기
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# 전처리: 0~1 범위로 정규화 및 채널 차원 추가
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0
x_train = x_train[..., np.newaxis]  # (60000, 28, 28, 1)
x_test  = x_test[..., np.newaxis]   # (10000, 28, 28, 1)

print("훈련 데이터:", x_train.shape)
print("테스트 데이터:", x_test.shape)

# 샘플 이미지 확인
fig, axes = plt.subplots(1, 10, figsize=(15, 2))
for i in range(10):
    axes[i].imshow(x_train[i, :, :, 0], cmap='gray')
    axes[i].set_title(str(y_train[i]))
    axes[i].axis('off')
plt.suptitle("MNIST 샘플 이미지")
plt.tight_layout()
plt.show()
```

2단계: CNN 모델 구축

```
from tensorflow.keras import layers, models

def build_cnn_model():
    model = models.Sequential([
        # 합성곱 블록 1
        layers.Conv2D(32, (3, 3), activation='relu', padding='same',
                      input_shape=(28, 28, 1)),
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # 합성곱 블록 2
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # 분류부
        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(10, activation='softmax')  # 10개 숫자 분류
    ])
    return model

model = build_cnn_model()
model.summary()
```

3단계: 모델 컴파일 및 학습

```
# 모델 컴파일
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 학습 (약 5분 소요)
history = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)
```

4단계: 결과 평가 및 시각화

```
# 테스트 정확도 평가
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"\n테스트 정확도: {test_acc:.4f} ({test_acc*100:.2f}%)")

# 학습 곡선 시각화
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# 정확도 곡선
axes[0].plot(history.history['accuracy'],     label='훈련 정확도')
axes[0].plot(history.history['val_accuracy'], label='검증 정확도')
axes[0].set_title('모델 정확도')
axes[0].set_xlabel('에포크')
axes[0].set_ylabel('정확도')
axes[0].legend()

# 손실 곡선
axes[1].plot(history.history['loss'],     label='훈련 손실')
axes[1].plot(history.history['val_loss'], label='검증 손실')
axes[1].set_title('모델 손실')
axes[1].set_xlabel('에포크')
axes[1].set_ylabel('손실')
axes[1].legend()

plt.tight_layout()
plt.show()
```

5단계: 예측 결과 확인

```
import numpy as np

# 테스트 이미지 예측
predictions = model.predict(x_test[:20])
predicted_labels = np.argmax(predictions, axis=1)

# 시각화
fig, axes = plt.subplots(2, 10, figsize=(20, 4))
for i in range(20):
    row, col = divmod(i, 10)
    axes[row, col].imshow(x_test[i, :, :, 0], cmap='gray')
    color = 'green' if predicted_labels[i] == y_test[i] else 'red'
    axes[row, col].set_title(
        f"예측:{predicted_labels[i]}\n정답:{y_test[i]}",
        color=color, fontsize=8
    )
    axes[row, col].axis('off')

plt.suptitle("CNN 예측 결과 (초록=정답, 빨강=오답)")
plt.tight_layout()
plt.show()
```

> 💡 *이 실습에서 잘 구성된 CNN 모델은 MNIST에서 **99% 이상의 정확도**를 달성할 수 있습니다. 에포크 수나 필터 수를 바꿔가며 정확도 변화를 직접 실험해 보세요.*

CIFAR-10으로 확장 실습 (심화)

```
# CIFAR-10: 10종류 컬러 이미지 (비행기, 자동차, 새, 고양이 등)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0

class_names = ['비행기','자동차','새','고양이','사슴',
               '개','개구리','말','배','트럭']

# 샘플 확인
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]], fontsize=7)
    plt.axis('off')
plt.suptitle("CIFAR-10 샘플")
plt.tight_layout()
plt.show()

print("CIFAR-10 훈련 데이터 shape:", x_train.shape)  # (50000, 32, 32, 3)
```

> ⚠️ *CIFAR-10은 MNIST보다 훨씬 복잡한 데이터셋으로, 단순한 CNN으로는 70~80% 정도의 정확도를 달성합니다. 더 높은 성능을 원한다면 ResNet 같은 깊은 모델이나 데이터 증강 기법을 사용해보세요.*

마치며

이번 챕터에서는 이미지 처리에 특화된 딥러닝 구조인 CNN을 배웠습니다.

CNN은 합성곱 연산을 통해 이미지의 공간적 특성을 효율적으로 학습하고, 필터, 스트라이드, 패딩, 풀링이라는 핵심 구성 요소를 결합하여 강력한 시각 지능을 구현합니다. LeNet에서 시작된 CNN은 AlexNet의 딥러닝 혁명, VGG의 깊이 탐구, ResNet의 잔차 연결 혁신을 거쳐 현재도 이미지 분류, 객체 탐지, 얼굴 인식 등 수많은 분야의 핵심 기술로 활용되고 있습니다.

📌 *Chapter 5 핵심 요약*

- *이미지는 픽셀값의 2D(흑백) 또는 3D(컬러) 배열로 표현됩니다.*
- ***합성곱 연산**은 필터를 슬라이딩하면서 지역적 특징을 추출합니다.*
- ***필터**는 특징 추출기, **스트라이드**는 이동 간격, **패딩**은 크기 유지 기법입니다.*
- ***특성맵**은 합성곱 결과, **풀링**은 크기를 줄이고 불변성을 높이는 연산입니다.*
- *CNN은 합성곱층 → 활성화 → 풀링층을 반복한 후 완전연결층으로 분류합니다.*
- *CNN은 LeNet(1998) → AlexNet(2012) → VGG(2014) → ResNet(2015) 순으로 발전하였습니다.*
- *CNN은 이미지 분류, 객체 탐지, 얼굴 인식 등 시각 지능의 핵심 기술입니다.*

💡 *다음 챕터 예고*

*Chapter 6에서는 **RNN(순환 신경망)**과 **자연어 처리(NLP)**를 다룹니다. CNN이 이미지의 공간적 구조를 다루었다면, RNN은 시간적 순서가 있는 데이터(텍스트, 음성, 시계열)를 처리하는 방법을 제공합니다. "오늘 날씨가 __"라는 문장에서 빈칸을 채우는 방법, 번역과 감성 분석의 원리를 배워봅시다.*

들어가며

앞서 Chapter 5에서는 이미지처럼 공간적 구조를 가진 데이터를 다루는 CNN을 배웠습니다. 그런데 우리가 일상에서 접하는 데이터 중에는 **순서(order)**가 매우 중요한 것들이 있습니다. 예를 들어 "나는 밥을 먹었다"와 "밥은 나를 먹었다"는 같은 단어로 이루어져 있지만 전혀 다른 의미를 가집니다. 시간의 흐름에 따른 주가 데이터, 한 단어씩 이어지는 문장, 초마다 변하는 음성 신호 — 이 모든 것이 **시퀀스(sequence)** 데이터입니다.

이번 챕터에서는 이러한 시퀀스 데이터를 처리하기 위해 설계된 **RNN(Recurrent Neural Network, 순환 신경망)**을 학습합니다. RNN이 어떻게 "기억"을 유지하는지, 그리고 그 한계를 극복하기 위해 등장한 **LSTM**과 **GRU**도 함께 살펴봅니다. 나아가 텍스트 데이터를 신경망이 이해할 수 있는 형태로 변환하는 **자연어 처리(NLP)** 기초 개념까지 익히겠습니다.

학습 목표

- 시퀀스 데이터의 개념과 종류를 설명할 수 있습니다.
- RNN의 순환 구조와 동작 원리를 이해할 수 있습니다.
- 기울기 소실 문제가 무엇인지, 왜 발생하는지 설명할 수 있습니다.
- LSTM과 GRU의 핵심 아이디어를 비교하여 설명할 수 있습니다.
- 텍스트 전처리, 토큰화, 워드 임베딩의 개념을 이해할 수 있습니다.
- Keras를 사용하여 간단한 텍스트 감성 분석 모델을 구현할 수 있습니다.

6.1 시퀀스 데이터란 무엇인가

시퀀스 데이터(Sequence Data)란 순서가 있는 데이터를 말합니다. 각 데이터 포인트가 독립적으로 존재하는 것이 아니라, 앞뒤 맥락과 연결되어 의미를 가집니다.

기존의 완전 연결 신경망(Fully Connected Network)이나 CNN은 입력 데이터 간의 순서를 고려하지 않습니다. 이미지의 픽셀 배열은 서로의 위치 관계가 있지만, 시간 축을 따라 변화하는 데이터의 흐름은 처리하기 어렵습니다. 이것이 RNN이 등장한 배경입니다.

시퀀스 데이터의 주요 종류

종류	예시	특징
텍스트(Text)	문장, 뉴스, 소설	단어·문자 단위의 순서가 의미를 결정
시계열(Time Series)	주가, 기온, 심전도	시간 간격에 따른 연속 수치 변화
음성(Speech)	음성 파일, 음악	시간에 따른 음파(파형)의 연속 변화
동영상(Video)	영화, 감시 카메라	시간 순서가 있는 이미지 프레임의 연속
DNA 서열	유전자 코드	A·T·G·C 염기의 순서

> 💡 ***핵심 포인트**: 시퀀스 데이터의 핵심은 "순서"입니다. 순서가 바뀌면 데이터의 의미가 달라집니다. 텍스트에서 단어의 순서를 바꾸면 문장의 의미가 달라지고, 시계열에서 과거와 미래가 뒤바뀌면 아무 의미가 없게 됩니다.*

시퀀스 데이터가 왜 어려운가

일반적인 신경망은 고정된 크기의 입력을 받습니다. 예를 들어 MNIST는 항상 28×28 픽셀 이미지를 입력으로 받습니다. 하지만 시퀀스 데이터는 길이가 가변적입니다. "안녕"은 2개의 단어이고, "오늘 날씨가 정말 좋네요"는 6개의 단어입니다. 기존 신경망 구조로는 이처럼 가변 길이 입력을 자연스럽게 처리하기 어렵습니다. RNN은 이 문제를 우아하게 해결합니다.

6.2 RNN의 순환 구조와 동작 원리

RNN(Recurrent Neural Network, 순환 신경망)은 시퀀스의 각 요소를 순서대로 처리하면서, 이전 단계의 정보를 다음 단계로 전달하는 신경망입니다.

순환 구조의 핵심 아이디어

일반 신경망은 입력 → 은닉층 → 출력의 단방향 흐름을 가집니다. RNN은 여기에 **자기 자신으로 돌아오는 연결(순환 연결)**을 추가합니다. 이 덕분에 이전 시점의 정보를 현재 시점에서 활용할 수 있습니다.

RNN의 핵심은 **은닉 상태(Hidden State)** `h_t` 입니다. 이것이 RNN의 "기억"에 해당합니다.

```
h_t = tanh(W_h * h_{t-1} + W_x * x_t + b)
```

- `x_t` : 현재 시점(t)의 입력
- `h_{t-1}` : 이전 시점(t-1)의 은닉 상태 (이전 기억)
- `h_t` : 현재 시점(t)의 은닉 상태 (현재 기억)
- `W_h`, `W_x`, `b` : 학습되는 가중치와 편향
- `tanh` : 활성화 함수 (출력 범위: -1 ~ 1)

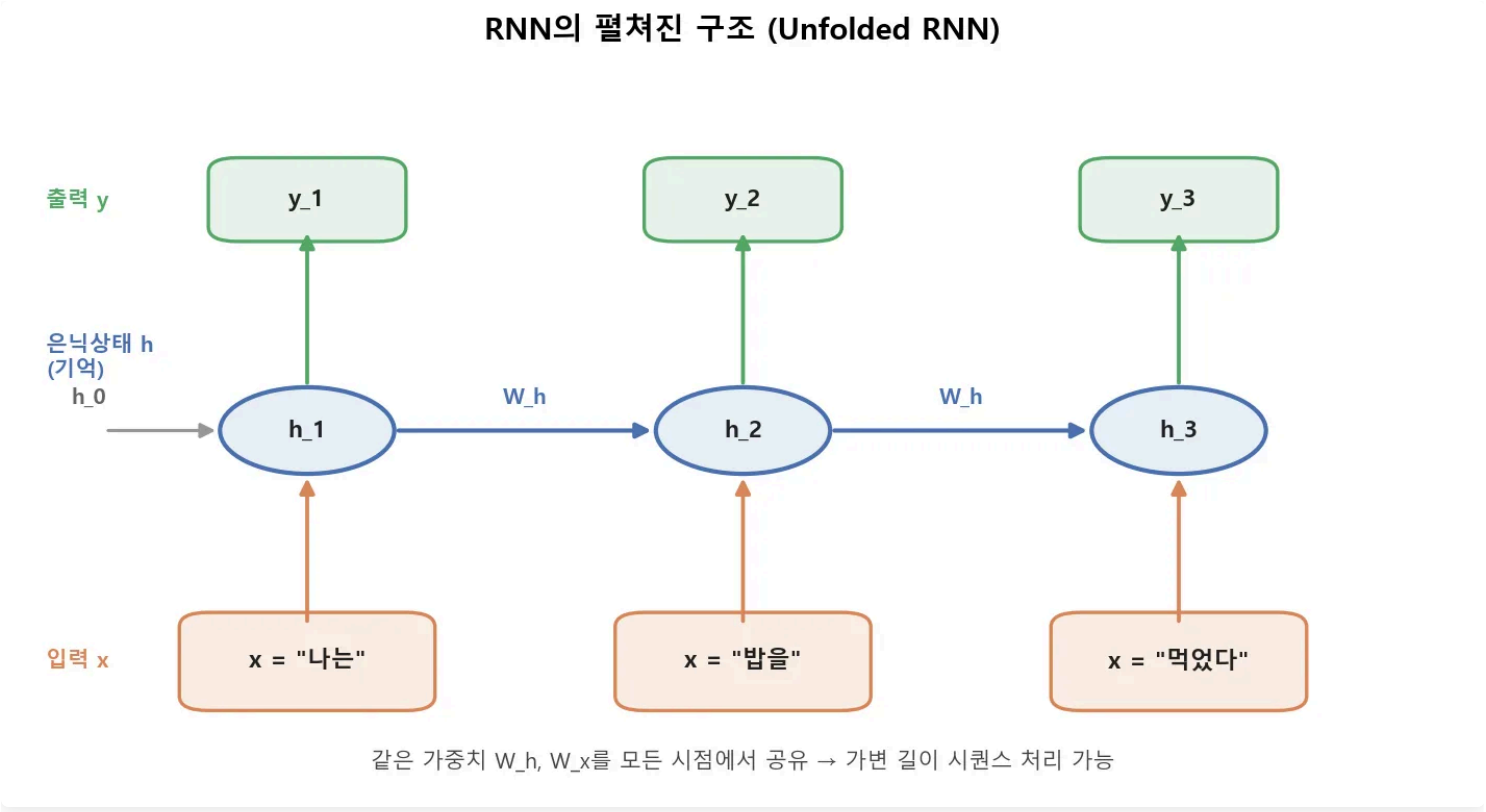
> 📌 ***중요**: RNN에서 가중치 `W_h` 와 `W_x` 는 **모든 시점에서 동일하게 공유**됩니다. 이것이 RNN이 가변 길이 시퀀스를 처리할 수 있는 이유입니다.*

RNN의 펼쳐진 구조 이해하기

RNN을 시간 축으로 펼쳐보면 각 시점마다 같은 가중치를 공유하는 신경망이 연결된 것처럼 이해할 수 있습니다.

예를 들어 "나는 밥을 먹었다"라는 문장을 처리할 때:

1. 시점 t=1: "나는" 입력 → h_1 생성
2. 시점 t=2: "밥을" 입력 + h_1 → h_2 생성
3. 시점 t=3: "먹었다" 입력 + h_2 → h_3 생성
4. 최종 h_3이 문장 전체의 의미를 담은 표현이 됩니다.



같은 가중치 W_h, W_x를 모든 시점에서 공유 → 가변 길이 시퀀스 처리 가능

이처럼 RNN은 문장을 한 단어씩 읽으면서 문맥을 누적해 나갑니다.

Keras로 RNN 구현해보기

```
import tensorflow as tf
from tensorflow import keras

# 간단한 SimpleRNN 예시
model = keras.Sequential([
    # input_shape = (시퀀스 길이, 특성 수)
    keras.layers.SimpleRNN(units=64, input_shape=(10, 1)),
    keras.layers.Dense(1)
])

model.summary()
```

위 코드에서 `units=64`는 은닉 상태 벡터의 차원(크기)을 의미합니다. 이 값이 클수록 더 복잡한 패턴을 학습할 수 있지만, 계산 비용도 증가합니다.

RNN의 다양한 입출력 구조

RNN은 입력과 출력의 형태에 따라 다양한 문제를 풀 수 있습니다.

구조	설명	활용 예시
일대다 (one-to-many)	하나의 입력 → 시퀀스 출력	이미지 캡션 생성
다대일 (many-to-one)	시퀀스 입력 → 하나의 출력	감성 분석, 문서 분류
다대다 (many-to-many)	시퀀스 입력 → 시퀀스 출력	기계 번역, 챗봇

6.3 기울기 소실(Vanishing Gradient) 문제

RNN은 이론적으로 아무리 긴 시퀀스도 처리할 수 있습니다. 하지만 실제로는 **긴 시퀀스를 다룰 때 심각한 문제**가 발생합니다. 바로 **기울기 소실(Vanishing Gradient)** 문제입니다.

기울기 소실이란

신경망은 역전파(Backpropagation)를 통해 학습됩니다. RNN에서는 오류 신호가 현재 시점에서 과거 시점으로 거슬러 올라가야 합니다. 이 과정을 **BPTT(Backpropagation Through Time, 시간을 통한 역전파)**라고 합니다.

문제는 역전파 과정에서 기울기가 시점을 거슬러 올라갈수록 점점 작아진다는 것입니다. tanh 함수의 미분값은 최대 1이기 때문에, 시점이 늘어날수록 곱해지는 값이 누적되어 기울기가 거의 0에 가까워집니다.

```
기울기 = ∂L/∂W ∝ (∂h_t/∂h_{t-1}) × (∂h_{t-1}/∂h_{t-2}) × ... × (∂h_2/∂h_1)
            = tanh'( ... ) × tanh'( ... ) × ... × tanh'( ... )
            ≈ (0.9) × (0.9) × ... → 0 (시점이 길어질수록)
```

> ⚠️ ***기울기 소실의 결과**: 먼 과거의 정보가 현재 학습에 거의 영향을 주지 못합니다. 즉, RNN은 **긴 의존성(Long-term Dependency)**을 학습하는 데 취약합니다.*

긴 의존성 문제의 예시

"저는 어릴 때부터 피아노를 배웠고, 학교에서도 음악 수업을 열심히 들었으며, 대학에서도 음악을 전공했습니다. 그래서 저는 음악 ___에 취직했습니다."

빈칸을 채우려면 앞의 내용 전체(피아노, 음악 수업, 음악 전공)를 기억하고 있어야 합니다. 하지만 기울기 소실로 인해 RNN은 먼 앞의 문맥을 학습하기 어렵습니다.

기울기 폭발(Exploding Gradient)

반대로 기울기가 폭발적으로 커지는 **기울기 폭발(Exploding Gradient)** 문제도 발생할 수 있습니다. 이는 **기울기 클리핑(Gradient Clipping)**으로 어느 정도 해결할 수 있습니다.

```
# Keras에서 기울기 클리핑 적용
optimizer = keras.optimizers.Adam(clipnorm=1.0)  # 기울기의 노름(norm)을 1.0으로 제한
```

기울기 소실 문제를 근본적으로 해결하기 위해 등장한 것이 바로 LSTM과 GRU입니다.

6.4 LSTM (Long Short-Term Memory)

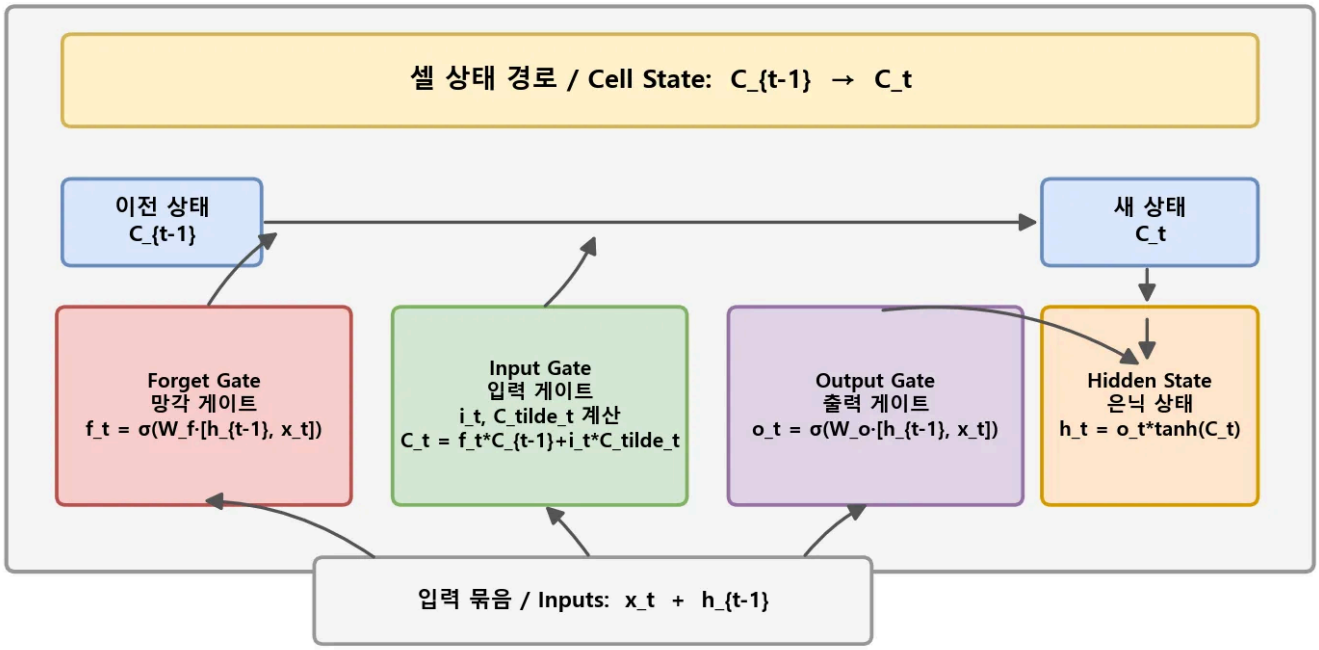
LSTM(Long Short-Term Memory, 장단기 메모리)은 1997년 Hochreiter와 Schmidhuber가 제안한 RNN의 개선 구조입니다. 이름 그대로 **짧은 기억(Short-Term)과 긴 기억(Long-Term)**을 모두 관리할 수 있도록 설계되었습니다.

LSTM의 핵심 아이디어: 셀 상태와 게이트

LSTM은 기존 RNN의 은닉 상태(`h_t`) 외에 **셀 상태(Cell State, `C_t`)**를 추가로 도입합니다. 셀 상태는 컨베이어 벨트처럼 정보가 거의 손실 없이 흘러갈 수 있는 통로 역할을 합니다.

정보의 흐름은 세 가지 **게이트(Gate)**로 정밀하게 제어됩니다.

LSTM 셀 구조



세 가지 게이트

1. 망각 게이트 (Forget Gate)

이전 셀 상태에서 어떤 정보를 버릴지 결정합니다. 시그모이드 함수를 통해 0(완전히 잊음)에서 1(완전히 기억) 사이의 값을 출력합니다.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. 입력 게이트 (Input Gate)

새로운 정보 중 얼마나 셀 상태에 추가할지 결정합니다.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

3. 출력 게이트 (Output Gate)

셀 상태를 바탕으로 현재 은닉 상태(출력)를 결정합니다.

```
o_t = σ(W_o · [h_{t-1}, x_t] + b_o)
h_t = o_t * tanh(C_t)
```

> 💡 ***게이트의 직관적 이해***: *게이트는 정보의 흐름을 조절하는 "밸브"입니다. 망각 게이트는 "지울 것을 결정"하고, 입력 게이트는 "기억할 것을 결정"하며, 출력 게이트는 "내보낼 것을 결정"합니다. 이 세 개의 밸브 덕분에 LSTM은 훨씬 긴 의존성을 학습할 수 있습니다.*

Keras로 LSTM 구현하기

```
import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([
    # LSTM 레이어 - SimpleRNN과 사용법은 동일합니다
    keras.layers.Embedding(input_dim=10000, output_dim=128),
    keras.layers.LSTM(units=64, return_sequences=False),
    keras.layers.Dense(1, activation='sigmoid')  # 이진 분류
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

`return_sequences=True`로 설정하면 각 시점의 은닉 상태를 모두 출력하고, `False`(기본값)로 설정하면 마지막 시점의 은닉 상태만 출력합니다. LSTM을 여러 층으로 쌓을 때는 마지막 층을 제외한 모든 층에 `return_sequences=True`를 설정해야 합니다.

6.5 GRU (Gated Recurrent Unit)

GRU(Gated Recurrent Unit)는 2014년 Cho 등이 제안한 LSTM의 경량화 버전입니다. LSTM의 셀 상태와 은닉 상태를 하나로 합치고, 세 개의 게이트를 두 개로 줄여 구조를 단순화했습니다.

GRU의 두 가지 게이트

1. 리셋 게이트 (Reset Gate)

이전 은닉 상태를 얼마나 반영할지 결정합니다. 0에 가까울수록 과거 정보를 무시하고 새 입력에 집중합니다.

```
r_t = σ(W_r · [h_{t-1}, x_t])
```

2. 업데이트 게이트 (Update Gate)

이전 은닉 상태와 새로운 후보 상태를 얼마나 혼합할지 결정합니다. LSTM의 망각 게이트와 입력 게이트를 하나로 합친 역할을 합니다.

```
z_t = σ(W_z · [h_{t-1}, x_t])
h̃_t = tanh(W · [r_t * h_{t-1}, x_t])
h_t = (1 - z_t) * h_{t-1} + z_t * h̃_t
```

LSTM vs GRU 비교

항목	LSTM	GRU
상태 수	셀 상태 + 은닉 상태 (2개)	은닉 상태 (1개)
게이트 수	3개 (망각, 입력, 출력)	2개 (리셋, 업데이트)
파라미터 수	많음	적음
학습 속도	상대적으로 느림	상대적으로 빠름
성능	데이터가 많을 때 유리	데이터가 적을 때 유리
장기 의존성	우수	우수 (LSTM과 유사)

> 📌 ***실무 팁**: 어떤 것을 선택할지 고민된다면 GRU부터 시작하는 것을 권장합니다. GRU는 LSTM보다 계산 비용이 낮으면서도 많은 경우에 비슷한 성능을 냅니다. 데이터가 충분히 많고 더 복잡한 패턴을 학습해야 한다면 LSTM을 선택합니다.*

Keras로 GRU 구현하기

```
# GRU는 LSTM과 사용법이 거의 동일합니다
model = keras.Sequential([
    keras.layers.Embedding(input_dim=10000, output_dim=128),
    keras.layers.GRU(units=64),
    keras.layers.Dense(1, activation='sigmoid')
])
```

6.6 자연어 처리(NLP) 기초 - 텍스트 전처리, 토큰화

자연어 처리(NLP, Natural Language Processing)는 인간이 사용하는 언어(자연어)를 컴퓨터가 이해하고 처리할 수 있도록 하는 기술입니다. 텍스트 데이터를 신경망에 입력하려면 숫자로 변환해야 하는데, 그 과정이 바로 텍스트 전처리입니다.

텍스트 전처리 파이프라인

텍스트 데이터를 모델에 입력하기까지의 일반적인 과정은 다음과 같습니다.



(1) 정제 (Cleaning)

텍스트에서 불필요한 노이즈를 제거합니다.

```
import re

text = "안녕하세요!!! 오늘 날씨가 정말 좋네요 ^^  #날씨 http://example.com"

# HTML 태그 제거
text = re.sub(r'<[^>]+>', '', text)

# URL 제거
text = re.sub(r'http\S+', '', text)

# 특수문자 제거 (한글, 영문, 숫자, 공백만 유지)
text = re.sub(r'[^가-힣a-zA-Z0-9\s]', '', text)

# 연속 공백 정리
text = re.sub(r'\s+', ' ', text).strip()

print(text)
# 출력: 안녕하세요 오늘 날씨가 정말 좋네요  날씨
```

(2) 토큰화 (Tokenization)

토큰화(Tokenization)는 텍스트를 의미 있는 최소 단위인 **토큰(Token)**으로 나누는 과정입니다.

단어 토큰화 (Word Tokenization)

```
# 영어 - 공백 기준으로 간단히 분리
sentence = "I love deep learning"
tokens = sentence.split()
print(tokens)
# ['I', 'love', 'deep', 'learning']

# 한국어 - KoNLPy 사용 (형태소 분석)
from konlpy.tag import Okt

okt = Okt()
sentence = "나는 딥러닝을 공부합니다"
tokens = okt.morphs(sentence)
print(tokens)
# ['나', '는', '딥', '러닝', '을', '공부', '합니다']

# 품사 태깅
print(okt.pos(sentence))
# [('나', 'Noun'), ('는', 'Josa'), ('딥', 'Noun'), ...]
```

> 💡 ***한국어 토큰화의 특수성**: 한국어는 교착어(膠着語)로 조사, 어미 등이 단어에 붙어 형태가 변합니다. "먹다", "먹었다", "먹겠다"는 모두 "먹-"이라는 어간을 공유합니다. 따라서 한국어 NLP에서는 형태소 분석기(KoNLPy, Mecab 등)를 사용하는 것이 일반적입니다.*

(3) 불용어 제거 (Stopword Removal)

불용어(Stopword)는 분석에 크게 기여하지 않는 단어입니다. 한국어의 경우 "은", "는", "이", "가" 같은 조사나 "그리고", "하지만" 같은 접속사가 해당됩니다.

```
stopwords = ['은', '는', '이', '가', '을', '를', '의', '에', '에서']

sentence = "나는 딥러닝을 공부합니다"
tokens = okt.morphs(sentence)
filtered = [word for word in tokens if word not in stopwords]

print(filtered)
# ['나', '딥러닝', '공부합니다']
```

(4) 정수 인코딩 (Integer Encoding)

각 단어에 고유한 정수를 매핑하여 단어장(Vocabulary)을 만듭니다.

```
from tensorflow.keras.preprocessing.text import Tokenizer

sentences = [
    "나는 딥러닝을 공부합니다",
    "딥러닝은 정말 재미있습니다",
    "나는 인공지능을 좋아합니다"
]

tokenizer = Tokenizer(num_words=100)  # 상위 100개 단어만 사용
tokenizer.fit_on_texts(sentences)

# 단어 인덱스 확인
print(tokenizer.word_index)
# {'나는': 1, '딥러닝을': 2, '딥러닝은': 3, ...}

# 텍스트를 정수 시퀀스로 변환
sequences = tokenizer.texts_to_sequences(sentences)
print(sequences)
# [[1, 2, 3], [4, 5, 6], ...]
```

(5) 패딩 (Padding)

신경망은 배치 단위로 처리하기 때문에 모든 시퀀스의 길이를 동일하게 맞춰야 합니다. **패딩(Padding)**은 짧은 시퀀스에 0을 채워 길이를 맞추는 작업입니다.

```
from tensorflow.keras.preprocessing.sequence import pad_sequences

sequences = [[1, 2, 3], [4, 5, 6, 7, 8], [9, 10]]

# maxlen=5로 패딩 (기본: 앞에 0 추가)
padded = pad_sequences(sequences, maxlen=5, padding='post')
print(padded)
# [[ 1  2  3  0  0]
#  [ 4  5  6  7  8]
#  [ 9 10  0  0  0]]
```

`padding='post'`는 뒤에 0을 추가하고, `padding='pre'`(기본값)는 앞에 0을 추가합니다.

6.7 워드 임베딩 - One-Hot Encoding, Word2Vec, GloVe

단어를 숫자로 표현하는 방법은 여러 가지입니다. 어떤 표현 방법을 사용하느냐에 따라 모델의 성능이 크게 달라집니다.

One-Hot Encoding

원-핫 인코딩(One-Hot Encoding)은 가장 단순한 단어 표현 방식입니다. 단어 수만큼의 차원을 가진 벡터에서, 해당 단어의 위치만 1이고 나머지는 모두 0인 벡터를 사용합니다.

```
# 단어장: {'나': 0, '는': 1, '딥러닝': 2, '좋아': 3}

나     = [1, 0, 0, 0]
는     = [0, 1, 0, 0]
딥러닝 = [0, 0, 1, 0]
좋아   = [0, 0, 0, 1]
```

원-핫 인코딩의 문제점:

1. **차원의 저주**: 단어 수만큼 벡터 차원이 커집니다. 영어 어휘는 수십만 개이므로 매우 비효율적입니다.
2. **의미 관계 표현 불가**: "고양이"와 "강아지"는 모두 반려동물이지만, 원-핫 벡터 간의 거리는 "고양이"와 "자동차"의 거리와 동일합니다.

> ⚠️ ***원-핫 인코딩의 한계**: 모든 단어 쌍의 코사인 유사도가 0입니다. 즉, 단어들 사이의 의미적 유사성을 전혀 표현하지 못합니다.*

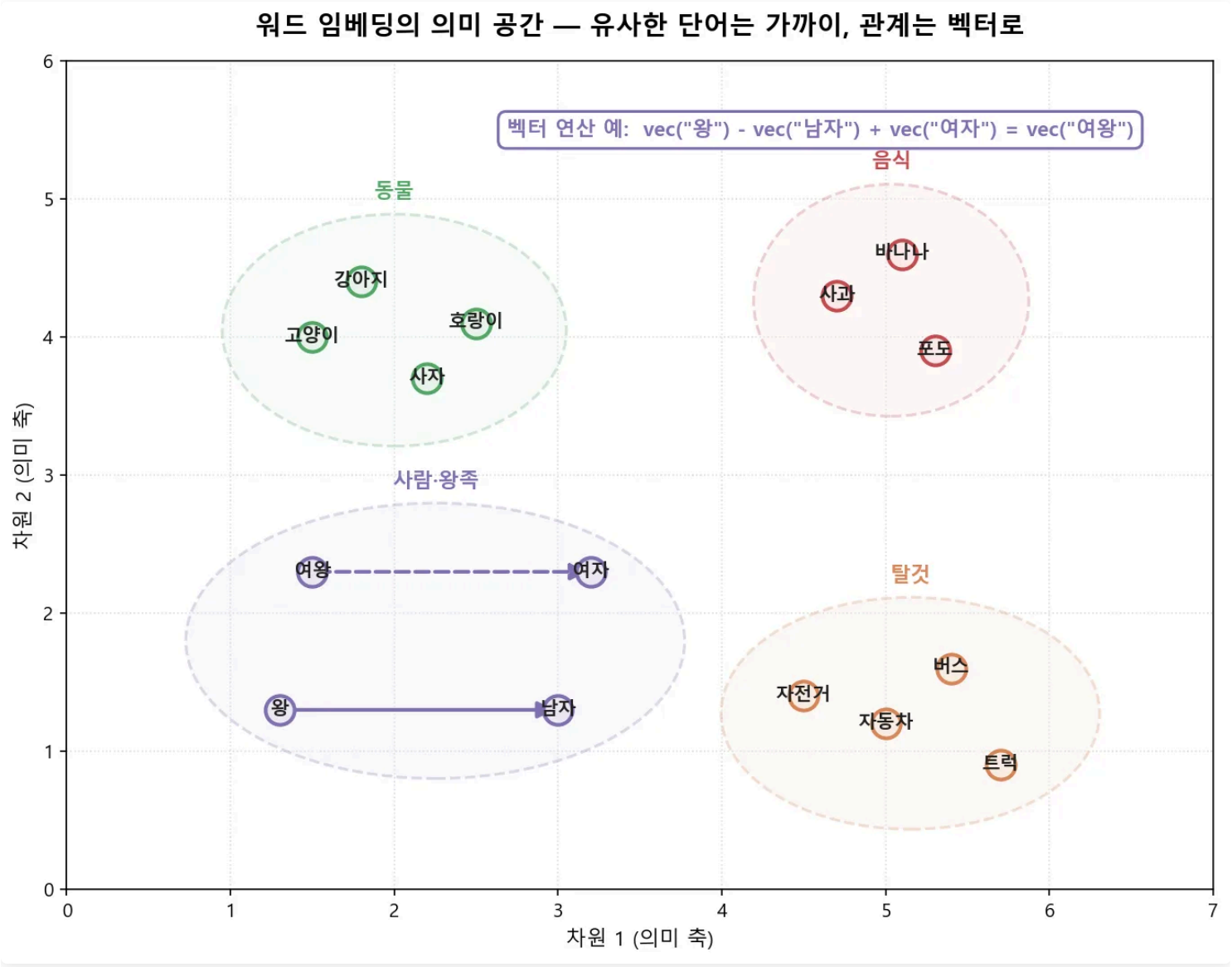
워드 임베딩 (Word Embedding)

워드 임베딩(Word Embedding)은 단어를 낮은 차원의 밀집 벡터(Dense Vector)로 표현하는 방법입니다. 의미적으로 유사한 단어는 비슷한 벡터를 가지도록 학습됩니다.

```
원-핫 인코딩: "고양이" → [0, 0, 1, 0, 0, ..., 0]  (50,000차원)
워드 임베딩:  "고양이" → [0.23, -0.15, 0.87, 0.41]  (4차원 예시)
```

가장 유명한 특성: 벡터 연산으로 의미 관계를 표현합니다.

```
"왕" - "남자" + "여자" ≈ "여왕"
```



Word2Vec

Word2Vec은 2013년 Google의 Mikolov 팀이 개발한 워드 임베딩 방법입니다. "주변 단어를 보면 그 단어의 의미를 알 수 있다"는 분포 가설(Distributional Hypothesis)에 기반합니다.

Word2Vec에는 두 가지 학습 방식이 있습니다.

방식	설명	특징
CBOW (Continuous Bag of Words)	주변 단어들로 중심 단어를 예측	빠른 학습, 자주 등장하는 단어에 유리
Skip-gram	중심 단어로 주변 단어들을 예측	느린 학습, 드문 단어에 더 유리

```
# Gensim을 사용한 Word2Vec 학습
from gensim.models import Word2Vec

# 학습 데이터 (단어 리스트의 리스트)
sentences = [
    ['나', '는', '딥러닝', '공부', '합니다'],
    ['딥러닝', '은', '인공지능', '분야', '입니다'],
    ['고양이', '와', '강아지', '는', '반려동물', '입니다']
]

# Word2Vec 학습
model = Word2Vec(
    sentences=sentences,
    vector_size=100,  # 임베딩 차원
    window=5,         # 주변 단어 범위
    min_count=1,      # 최소 등장 횟수
    sg=1              # 0=CBOW, 1=Skip-gram
)

# 유사 단어 찾기
similar_words = model.wv.most_similar('딥러닝', topn=3)
print(similar_words)

# 단어 벡터 확인
print(model.wv['딥러닝'])  # 100차원 벡터
```

GloVe (Global Vectors for Word Representation)

GloVe는 2014년 스탠퍼드 대학에서 개발한 워드 임베딩 방법입니다. Word2Vec이 로컬 문맥 창(window)만 보는 것과 달리, GloVe는 전체 문서에서의 **단어 공출현 빈도(Co-occurrence)** 행렬을 활용합니다.

비교 항목	Word2Vec	GloVe
학습 방식	신경망 기반 예측 모델	행렬 분해(Matrix Factorization)
활용 정보	로컬 문맥 (윈도우)	전역 통계 (공출현 행렬)
속도	빠름	느리지만 메모리 효율적
성능	유사	유사

실무에서는 Word2Vec, GloVe 외에도 대규모 언어 모델로 사전 학습된 **BERT**, **GPT** 등의 임베딩을 많이 활용합니다.

Keras Embedding 레이어

딥러닝 모델 내에서 워드 임베딩을 직접 학습시킬 수도 있습니다.

```
import tensorflow as tf
from tensorflow import keras

vocab_size = 10000   # 단어장 크기
embedding_dim = 128  # 임베딩 차원
max_length = 100     # 최대 시퀀스 길이

model = keras.Sequential([
    # Input 레이어로 입력 형태를 명시 (Keras 3 권장 방식)
    keras.layers.Input(shape=(max_length,)),
    # Embedding 레이어: (vocab_size) → (embedding_dim) 차원으로 변환
    keras.layers.Embedding(
        input_dim=vocab_size,
        output_dim=embedding_dim
    ),
    keras.layers.LSTM(64),
    keras.layers.Dense(1, activation='sigmoid')
])
```

> 📌 ***Keras 3 변경 사항***: *이전 버전에서는 `Embedding(input_length=...)` 인자로 입력 길이를 지정했지만, Keras 3부터는 deprecated되었습니다. 대신 위 예시처럼 `keras.layers.Input(shape=(max_length,))` 로 입력 형태를 먼저 선언하는 것이 권장됩니다.*

`Embedding` 레이어는 정수 인덱스를 받아 학습 가능한 임베딩 벡터로 변환합니다. 모델 학습 과정에서 태스크에 맞는 최적의 임베딩을 자동으로 학습합니다.

6.8 언어 모델의 개념 — RNN에서 Transformer로의 전환 배경

언어 모델(Language Model)이란 주어진 텍스트 시퀀스에서 다음 단어(또는 문장)가 무엇일지 예측하는 확률 모델입니다.

언어 모델의 역할

언어 모델은 다음과 같은 확률을 모델링합니다.

```
P("오늘 날씨가 ___") → "좋다" (높은 확률) vs "먹었다" (낮은 확률)
```

이를 수식으로 표현하면:

$$P(w_1, w_2, \ldots, w_n) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1,w_2) \times \ldots$$

언어 모델이 잘 작동하면 자연스러운 텍스트 생성, 기계 번역, 음성 인식, 자동 완성 등이 가능해집니다.

RNN 기반 언어 모델의 성과와 한계

RNN, LSTM, GRU를 활용한 언어 모델은 2010년대 중반까지 NLP의 표준이었습니다. 특히 **Seq2Seq(Sequence-to-Sequence)** 구조는 기계 번역 분야에서 큰 성과를 거뒀습니다.

하지만 RNN 계열 모델에는 근본적인 한계가 있었습니다.

1. 순차 처리의 비효율성

RNN은 시퀀스를 반드시 처음부터 끝까지 **순서대로** 처리해야 합니다. 이 때문에 병렬 처리(GPU 활용)가 어렵고, 긴 시퀀스에서 학습 속도가 매우 느립니다.

2. 장거리 의존성의 한계

LSTM/GRU가 기울기 소실 문제를 어느 정도 해결했지만, 수백 단어 이상의 매우 긴 문서에서는 여전히 초반 문맥을 효과적으로 전달하기 어렵습니다.

3. 어텐션 메커니즘의 등장

2015년 Bahdanau 등이 **어텐션 메커니즘(Attention Mechanism)**을 제안했습니다. 이는 디코더가 인코더의 모든 은닉 상태를 직접 참조하여, 현재 출력과 관련성이 높은 부분에 더 집중할 수 있게 해주는 기법입니다.

> 💡 ***어텐션의 직관**: 긴 영어 문장을 한국어로 번역할 때, 모든 단어를 동시에 고려하는 것이 아니라 현재 번역 중인 부분과 관련된 원문의 특정 단어에 더 집중합니다. 어텐션은 이 "집중"을 수학적으로 구현한 것입니다.*

Transformer로의 전환

2017년 Google이 발표한 논문 "Attention is All You Need"는 RNN을 완전히 제거하고 **어텐션 메커니즘만으로** 구성된 **Transformer** 구조를 제안했습니다. Transformer는:

- RNN 없이 **전체 시퀀스를 한 번에 병렬 처리**
- **Self-Attention**으로 시퀀스 내 모든 단어 간의 관계를 직접 계산
- GPU를 활용한 고속 학습 가능

Transformer의 등장은 NLP에 혁명을 가져왔고, 이를 기반으로 **BERT**, **GPT**, **T5** 등 현대 대규모 언어 모델(LLM)이 탄생했습니다.

모델 계보	특징	등장 시기
RNN / LSTM	순차 처리, 기울기 소실 한계	1986~1997
Seq2Seq + Attention	RNN + 어텐션으로 장거리 의존성 개선	2014~2015
Transformer	순수 어텐션 기반, 병렬 처리	2017
BERT / GPT	대규모 사전학습 언어 모델	2018~
GPT-4 / Claude	초대규모 언어 모델(LLM)	2022~

> 📌 ***다음 챕터 예고**: Transformer와 대규모 언어 모델의 원리는 Part 3(Chapter 7~)에서 본격적으로 다룹니다. 이번 챕터에서 배운 RNN과 임베딩 개념이 Transformer를 이해하는 기초가 됩니다.*

[실습] 간단한 텍스트 감성 분석

IMDB 영화 리뷰 데이터셋을 활용하여 긍정/부정을 분류하는 감성 분석 모델을 만들어 봅니다. Keras에 기본 내장된 데이터셋을 사용하므로 별도 다운로드가 필요 없습니다.

실습 환경

이 실습은 Google Colab에서 실행하는 것을 권장합니다. 다음 라이브러리가 필요합니다.

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
import matplotlib.pyplot as plt

print(f"TensorFlow 버전: {tf.__version__}")
```

Step 1: 데이터 로드

```
# 상위 10,000개 단어만 사용
VOCAB_SIZE = 10000
MAX_LEN = 200  # 최대 리뷰 길이

# IMDB 데이터셋 로드 (자동으로 정수 인코딩된 형태로 제공)
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=VOCAB_SIZE)

print(f"훈련 샘플 수: {len(x_train)}")   # 25,000
print(f"테스트 샘플 수: {len(x_test)}")   # 25,000
print(f"첫 번째 리뷰 (정수): {x_train[0][:10]}...")
print(f"레이블: {y_train[0]}")            # 1=긍정, 0=부정

# 리뷰 길이 분포 확인
lengths = [len(x) for x in x_train]
print(f"평균 리뷰 길이: {np.mean(lengths):.0f}")
print(f"최대 리뷰 길이: {np.max(lengths)}")
```

Step 2: 원문 확인 (선택 사항)

```
# 정수 → 단어 변환을 위한 딕셔너리
word_index = imdb.get_word_index()
reverse_word_index = {v: k for k, v in word_index.items()}

def decode_review(encoded_review):
    """정수 시퀀스를 텍스트로 변환합니다."""
    # 인덱스 3부터 실제 단어 (0=패딩, 1=시작, 2=미지어 단어)
    return ' '.join([reverse_word_index.get(i - 3, '?') for i in encoded_review])

print(decode_review(x_train[0][:20]))
```

Step 3: 데이터 전처리 (패딩)

```
# 시퀀스 길이를 MAX_LEN=200으로 통일
x_train_padded = pad_sequences(x_train, maxlen=MAX_LEN, padding='post', truncating='post')
x_test_padded = pad_sequences(x_test, maxlen=MAX_LEN, padding='post', truncating='post')

print(f"패딩 후 훈련 데이터 shape: {x_train_padded.shape}")  # (25000, 200)
print(f"패딩 후 테스트 데이터 shape: {x_test_padded.shape}")  # (25000, 200)
```

Step 4: 모델 구성 - LSTM 기반 감성 분류기

```
EMBEDDING_DIM = 64

model = keras.Sequential([
    # 0. Input 레이어: 입력 형태 명시 (Keras 3 권장)
    keras.layers.Input(shape=(MAX_LEN,), name='input'),

    # 1. 임베딩 레이어: 정수 → 64차원 밀집 벡터
    keras.layers.Embedding(
        input_dim=VOCAB_SIZE,
        output_dim=EMBEDDING_DIM,
        name='embedding'
    ),

    # 2. LSTM 레이어: 시퀀스에서 의미 추출
    keras.layers.LSTM(
        units=64,
        dropout=0.2,          # 입력 드롭아웃
        recurrent_dropout=0.2, # 순환 연결 드롭아웃
        name='lstm'
    ),

    # 3. 드롭아웃: 과적합 방지
    keras.layers.Dropout(0.5, name='dropout'),

    # 4. 출력 레이어: 이진 분류 (긍정/부정)
    keras.layers.Dense(1, activation='sigmoid', name='output')
])

model.summary()
```

Step 5: 모델 컴파일 및 학습

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',  # 이진 분류 손실 함수
    metrics=['accuracy']
)

# 검증 데이터를 일부 사용
history = model.fit(
    x_train_padded, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2,  # 훈련 데이터의 20%를 검증에 사용
    verbose=1
)
```

학습 중 출력 예시:

```
Epoch 1/5
157/157 [==============================] - 25s - loss: 0.5821 - accuracy: 0.6912 - val_loss:
0.3891 - val_accuracy: 0.8342
Epoch 2/5
157/157 [==============================] - 22s - loss: 0.3245 - accuracy: 0.8701 - val_loss:
0.3101 - val_accuracy: 0.8756
...
Epoch 5/5
157/157 [==============================] - 22s - loss: 0.2102 - accuracy: 0.9213 - val_loss:
0.2891 - val_accuracy: 0.8890
```

Step 6: 모델 평가

```
# 테스트 데이터로 최종 성능 평가
test_loss, test_accuracy = model.evaluate(x_test_padded, y_test, verbose=0)
print(f"테스트 손실: {test_loss:.4f}")
print(f"테스트 정확도: {test_accuracy:.4f}")  # 약 87~89% 예상
```

Step 7: 학습 곡선 시각화

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# 정확도 그래프
ax1.plot(history.history['accuracy'], label='훈련 정확도')
ax1.plot(history.history['val_accuracy'], label='검증 정확도')
ax1.set_title('모델 정확도')
ax1.set_xlabel('에폭')
ax1.set_ylabel('정확도')
ax1.legend()
ax1.grid(True)

# 손실 그래프
ax2.plot(history.history['loss'], label='훈련 손실')
ax2.plot(history.history['val_loss'], label='검증 손실')
ax2.set_title('모델 손실')
ax2.set_xlabel('에폭')
ax2.set_ylabel('손실')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.savefig('sentiment_training.png', dpi=100, bbox_inches='tight')
plt.show()
```

Step 8: 새 리뷰 예측해보기

```
def predict_sentiment(review_text, model, word_index, max_len=200):
    """새로운 영어 리뷰에 대한 감성을 예측합니다."""
    # 텍스트를 정수 시퀀스로 변환
    words = review_text.lower().split()
    sequence = [word_index.get(word, 2) + 3 for word in words]  # 2=미지어

    # 패딩
    padded = pad_sequences([sequence], maxlen=max_len, padding='post')

    # 예측
    score = model.predict(padded, verbose=0)[0][0]
    sentiment = "긍정 (Positive)" if score > 0.5 else "부정 (Negative)"

    return sentiment, score

# 긍정 리뷰 예시
positive_review = "This movie was absolutely wonderful and touching. Great acting and beautiful story."
sentiment, score = predict_sentiment(positive_review, model, word_index)
print(f"리뷰: {positive_review}")
print(f"예측: {sentiment} (점수: {score:.4f})")

print()

# 부정 리뷰 예시
negative_review = "This was the worst movie I have ever seen. Terrible acting and boring plot."
sentiment, score = predict_sentiment(negative_review, model, word_index)
print(f"리뷰: {negative_review}")
print(f"예측: {sentiment} (점수: {score:.4f})")
```

실습 결과 해석

> 📌 ***결과 확인 포인트*** *- 테스트 정확도가 약 87~89% 수준이면 정상적인 학습이 진행된 것입니다. - 훈련 정확도와 검증 정확도의 차이가 크다면 과적합이 발생한 것이므로 드롭아웃 비율을 높이거나 에폭 수를 줄여봅니다. - GRU로 교체 (* `keras.layers.GRU(64)` *)하여 성능을 비교해보세요.*

더 나아가기

```
# 양방향 LSTM (Bidirectional LSTM)으로 성능 향상 시도
model_bi = keras.Sequential([
    keras.layers.Input(shape=(MAX_LEN,)),
    keras.layers.Embedding(VOCAB_SIZE, EMBEDDING_DIM),
    # 순방향과 역방향 LSTM을 동시에 학습
    keras.layers.Bidirectional(keras.layers.LSTM(64, dropout=0.2)),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(1, activation='sigmoid')
])

model_bi.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# 양방향 LSTM은 일반적으로 단방향보다 더 좋은 성능을 냅니다
```

> 💡 ***양방향 LSTM(Bidirectional LSTM)***: *일반 LSTM은 앞에서 뒤로만 정보가 흐릅니다. 양방향 LSTM은 순방향과 역방향 두 LSTM을 동시에 사용하여 양쪽 문맥을 모두 고려합니다. "나는 ___ 에서 놀았다"에서 빈칸을 채울 때 앞뒤 문맥을 모두 참고하는 것이 더 정확한 것처럼, 양방향 접근이 대부분의 NLP 태스크에서 성능을 향상시킵니다.*

마치며

이번 챕터에서는 시퀀스 데이터를 처리하는 핵심 도구들을 학습했습니다.

핵심 내용 요약

주제	핵심 내용
시퀀스 데이터	순서가 중요한 데이터. 텍스트, 시계열, 음성 등
RNN	은닉 상태를 통해 이전 정보를 현재에 전달하는 순환 구조
기울기 소실	긴 시퀀스에서 역전파 기울기가 소실되어 장거리 의존성 학습 어려움
LSTM	셀 상태와 3개의 게이트(망각/입력/출력)로 기울기 소실 극복
GRU	LSTM의 경량화 버전. 2개의 게이트(리셋/업데이트)로 단순화
텍스트 전처리	정제 → 토큰화 → 정수 인코딩 → 패딩의 파이프라인
워드 임베딩	단어를 밀집 벡터로 표현. One-Hot의 한계를 극복한 Word2Vec, GloVe
언어 모델	RNN → Seq2Seq+어텐션 → Transformer → LLM으로의 발전

다음 챕터 미리보기

Chapter 7 (Part 3 시작): 생성형 AI의 개요에서는 본격적으로 AI가 새로운 콘텐츠를 "생성"하는 기술을 다룹니다. 이번 챕터에서 배운 RNN 기반의 언어 모델과 임베딩 개념이 Transformer와 대규모 언어 모델(LLM)을 이해하는 핵심 기초가 됩니다. 특히 어텐션 메커니즘이 어떻게 RNN의 한계를 극복하고, GPT/Claude 같은 현대 AI의 토대가 되었는지 살펴봅니다.